

UNIVERSIDAD DE MÁLAGA
ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

APLICACIÓN DE TÉCNICAS DE MACHINE
LEARNING PARA LA PREDICCIÓN PRECOZ DE
LA ENFERMEDAD DE ALZHEIMER

GRADO EN INGENIERÍA DE
SISTEMAS DE TELECOMUNICACIÓN

YUNES ABDELKADER ISMAEL

MÁLAGA, 2022

E.T.S De Ingeniería De Telecomunicación, Universidad de Málaga

APLICACIÓN DE TÉCNICAS DE MACHINE LEARNING PARA LA PREDICCIÓN PRECOZ DE LA ENFERMEDAD DEL ALZHEIMER

Autor: Yunes Abdelkader Ismael

Tutor: Jorge Munilla Fajardo

Departamento: Ingeniería de Comunicaciones

Titulación: Grado en Ingeniería de Sistemas de Telecomunicación

Palabras clave: diagnóstico de la enfermedad de Alzheimer, imágenes PET, Redes Neuronal Convolucional, Red Neuronal Recurrente, aprendizaje profundo, clasificación de imágenes

Resumen

La demencia más común entre personas mayores de 65 años es la enfermedad de Alzheimer. El Alzheimer es una enfermedad irreversible que deteriora progresivamente ciertas zonas del cerebro provocando una pérdida escalonada de la capacidad intelectual y funcional del paciente.

Actualmente, la enfermedad de Alzheimer no tiene cura, pero existen ciertos tratamientos que mejoran la calidad de vida del paciente. A pesar de no existir un tratamiento que actúe directamente sobre el Alzheimer, su diagnóstico precoz se ha convertido en uno de los principales focos de investigación en el campo de las enfermedades neurodegenerativas para el desarrollo de un tratamiento futuro en fases muy iniciales que consiga frenar la progresión de la enfermedad o, al menos, retrasar su aparición y sus síntomas.

Este Trabajo Fin de Grado implementa un software capaz de predecir la enfermedad de Alzheimer bajo el análisis de un conjunto de imágenes cerebrales PET a fin de asistir a un diagnóstico más preciso y reducir posibles errores que puedan surgir en la interpretación por parte de un médico especializado en la medicina nuclear.

Esto es posible gracias a la Inteligencia Artificial y, más concretamente, a las técnicas de Machine Learning que poseen la capacidad para aprender a diagnosticar la enfermedad de Alzheimer con un gran acierto.

APPLICATION OF MACHINE LEARNING TECHNIQUES FOR THE EARLY PREDICTION OF ALZHEIMER'S DISEASE

Author: Yunes Abdelkader Ismael

Tutor: Jorge Munilla Fajardo

Department: Communications Engineering

Title: Degree in Telecommunications Systems Engineering

Key words: diagnosis of Alzheimer's disease, PET images, Convolutional Neural Networks, Recurrent Neural Networks, Deep Learning, image classification

Abstract

The most common dementia among people over 65 is Alzheimer's disease. Alzheimer's is an irreversible disease that progressively deteriorates certain areas of the brain, causing a gradual loss of the patient's intellectual and functional capacity.

Currently, Alzheimer's disease has no cure, but there are certain treatments that improve the quality of life of the patient. Despite the absence of a treatment that acts directly on Alzheimer's disease, its early diagnosis has become one of the main research focuses in the field of neurodegenerative diseases for the development of a future treatment in very early stages that can slow its progression of the disease or, at least, delay its development and its symptoms.

This Final Degree Project implements a software capable of predicting Alzheimer's disease by analyzing a set of PET brain images in order to assist in a more precise diagnosis and reduce possible errors that may arise in interpretation by a specialized doctor in nuclear medicine.

This is possible thanks to Artificial Intelligence and, more specifically, to Machine Learning techniques that have the ability to learn to diagnose Alzheimer's disease with great accuracy.

Agradecimientos

Antes de comenzar con la exposición del Trabajo Fin de Grado, me gustaría mostrar mis agradecimientos a ciertas personas que han hecho posible y contribuido de algún modo a la realización de este TFG:

- En primer lugar, a mi tutor, Jorge Munilla, por brindarme la oportunidad de trabajar con él y adquirir todo el conocimiento necesario para abordar esta temática del Machine Learning.
- Por otro lado, a mi entorno familiar, especialmente a mis padres, por su apoyo incondicional y su constante dedicación en mi bienestar durante todo el transcurso de la asignatura y el trabajo.

Índice

Capítulo 1. Introducción	1
1.1 Contexto	1
1.2 Objetivos	2
Capítulo 2. Alzheimer	3
2.1 Causas del Alzheimer	4
2.2 Síntomas y Etapas de la Enfermedad	5
2.3 Tratamientos y Prevención	6
Capítulo 3. Machine Learning	7
3.1 Deep Learning	9
3.1.1 Red Neuronal Artificial	10
3.1.2 Red Neuronal Convolucional	20
3.1.3 Red Neuronal Recurrente	22
3.1.4 Librerías	27
3.2 Diagnóstico de la Habilidad de un modelo de ML	28
3.3. La Reproducibilidad de los algoritmos de Machine Learning	34
3.3.1 Validación Cruzada K-Fold	35
3.3.2. Ensamblados	37
Capítulo 4. Implementación del Clasificador	39
4.1. Conjunto de Datos	39
4.1.1 Tomografía por Emisión de Positrones	40
4.1.2 Base de Datos	41
4.2. Arquitectura de Red Propuesta	42
4.3. Procesamiento de las Imágenes cerebrales	44
4.4. Configuración del Modelo de Clasificación	46
Capítulo 5. Interpretación del rendimiento del clasificador	51
Capítulo 6. Conclusiones	57
Referencias	58

Acrónimos

AD	Alzheimer Disease
Adagrad	Adaptive Gradient Descent
ADI	Alzheimer's Disease International
ADNI	Alzheimer's Disease Neuroimaging Initiative
CN	Cognitive Normal
CNN	Convolutional Neural Network
CV	Cross Validation
FN	Falso Negativo
FP	Falso Positivo
GPU	Graphic Process Unit
GRU	Gates Recurrent Unit
ML	Machine Learning
MCI	Mild Cognitive Impairment
LSTM	Long Short Term Memory
ReLU	Rectified Linear Unit
RNA	Red Neuronal Artificial
RNN	Reccurrent Neural Network
TFG	Trabajo Fin de Grado
SGD	Stochastic Gradient Descent
PET	Positron Emission Tomography
VN	Verdadero Negativo
VP	Verdadero Positivo

Listado de Figuras

Figura 3.1. Base de Datos de la Calidad de Vino.	8
Figura 3.2. Estructura de la RNA.	10
Figura 3.3. Algoritmo Forward Propagation.	11
Figura 3.4. Comparación de un Modelo Lineal vs No-Lineal.	12
Figura 3.5. Función de Activación Sigmoidal.	13
Figura 3.6. Función de Activación Tangente Hiperbólica.....	14
Figura 3.7. Función de Activación ReLU.	14
Figura 3.8. Algoritmo BackPropagation.	16
Figura 3.9. Funcionamiento del Algoritmo del Gradiente Descendente.	17
Figura 3.10. Ejemplo de Aplicación de una CNN.	22
Figura 3.11. Estructura de la RNN.....	23
Figura 3.12. Comparación de una celda RNN vs LSTM.	24
Figura 3.13. Celda GRU.....	25
Figura 3.14. Arquitectura One to Many.	26
Figura 3.15. Arquitectura Many to One.....	26
Figura 3.16. Arquitectura Many to Many.....	27
Figura 3.17. Split Train-Test.	28
Figura 3.18. Curva de Aprendizaje de un Modelo Deep Learning con Sub-Ajuste.	29
Figura 3.19. Curva de Aprendizaje de un Modelo Deep Learning con Buen Ajuste. ...	29
Figura 3.20. Curva de Aprendizaje de un Modelo Deep Learning con Sobreajuste.....	30
Figura 3.21. Curva ROC.....	33
Figura 3.22. Técnica de la Validación Cruzada K-Fold.	36
Figura 3.23. Técnicas Simples de Conjunto.	38
Figura 4.1. Cortes en los diferentes planos de una imagen cerebral PET de un paciente con Alzheimer y un paciente con un cognitivo normal.	41
Figura 4.2. Diagrama de Flujo de la Arquitectura Propuesta.	43
Figura 4.3. Redimensión de la imagen a un tamaño inferior.	45
Figura 4.4. Arquitectura del Modelo de Clasificación.	47
Figura 4.5. Esquema del Modelo de Clasificación.	48
Figura 4.6. Curva de aprendizaje del modelo de clasificación.	50
Figura 5.1. Evaluación del Clasificador en un Grupo de Sujetos de Prueba.	51
Figura 5.2. Evaluación del Clasificador mediante la CV Estratificada 10-Fold.....	52
Figura 5.3. Evaluación del Conjunto de Clasificadores en un Grupo de Sujetos de Prueba.....	52
Figura 5.4. Evaluación del Conjunto de Clasificadores mediante la Validación Cruzada Estratificada 10-Fold.	53
Figura 5.5. Curva ROC del Conjunto de Clasificadores.....	54
Figura 5.6. Matriz de Confusión del Conjunto de Clasificadores	55

Listado de Tablas

Tabla 3.1. Matriz de Confusión..... 2

Tabla 4.1. Información Demográfica de las Imágenes Recopiladas. 42

Tabla 4.2. Aplicación de la Codificación One-Hot en las etiquetas. 46

Tabla 5.1. Rendimiento del Conjunto de Clasificadores..... 54

Capítulo 1. Introducción

1.1 Contexto

La demencia más común entre personas mayores de 65 años es la enfermedad de Alzheimer. El Alzheimer es una enfermedad irreversible que deteriora progresivamente ciertas zonas del cerebro provocando una pérdida escalonada de la capacidad intelectual y funcional del paciente [1].

Actualmente, la enfermedad de Alzheimer no tiene cura, pero existen ciertos tratamientos que mejoran la calidad de vida del paciente. A pesar de no existir un tratamiento que actúe directamente sobre el Alzheimer, su diagnóstico precoz se ha convertido en uno de los principales focos de investigación en el campo de las enfermedades neurodegenerativas para el desarrollo de un tratamiento futuro en fases muy iniciales que consiga frenar la progresión de la enfermedad o, al menos, retrasar su aparición y sus síntomas [2].

A mediados de los años 50 del siglo pasado, surgió el nacimiento de la Inteligencia Artificial (IA), no obstante, no ha sido hasta estos últimos años cuando el crecimiento de la Inteligencia Artificial ha crecido de manera exponencial y gran culpa de ello tiene que ver con la gran cantidad de información disponible, la gran capacidad de almacenamiento y la amplia disponibilidad de GPU que hacen que el procesamiento paralelo sea más rápido, más económico y potente [3]. Gracias a esta tecnología y, más específicamente, a las técnicas de Machine Learning es posible diseñar una herramienta software que asista y ayude a un médico a tomar un diagnóstico más rápido y preciso de la enfermedad de Alzheimer.

Al igual que un médico necesita aprender e identificar patrones bajo la experiencia de casos anteriores, las técnicas de Machine Learning realizan las predicciones en base al estudio del comportamiento en los datos observados [4].

1.2 Objetivos

El principal objetivo del proyecto consiste en probar si la aplicación de nuevos sistemas de Machine Learning, en concreto un modelo de una red neuronal híbrida, es capaz de aprender a detectar las características más influyentes de un conjunto de imágenes cerebrales PET a fin de mejorar la capacidad de predicción de la enfermedad de Alzheimer, lo cual es fundamental para el desarrollo de nuevos fármacos y tratamientos.

La habilidad de nuestro modelo de red neuronal híbrida para predecir correctamente la enfermedad de Alzheimer a través de imágenes cerebrales PET se medirá con una Validación Cruzada K-Fold y se optimizará mediante el uso combinado de predicciones de múltiples clasificadores (Ensamblados). Toda la implementación del código del modelo neuronal se hará en lenguaje Python y usando librerías de Código Abierto como Keras y Tensorflow, entre otras.

Con el fin de estructurar el Trabajo Fin de Grado, la temática de los siguientes capítulos que se desarrollarán a lo largo del proyecto, así como el orden establecido vendrá marcado como se detalla a continuación:

- La enfermedad de Alzheimer: ¿Qué es?, ¿Cuál es su origen?, ¿Cómo se manifiesta?, ¿Cómo se puede prevenir?
- Fundamentos teóricos del Machine Learning que serán necesarios posteriormente su aplicación en el siguiente capítulo para poder abordar la temática principal del presente TFG.
- Implementación de nuestro modelo de red neuronal para predecir la enfermedad de Alzheimer.
- Exposición de las conclusiones obtenidas en la investigación.
- Referencias bibliográficas dónde se ha extraído la información necesaria para desarrollar la memoria del TFG.

Capítulo 2. Alzheimer

Todos los pacientes diagnosticados con alguna demencia padecieron previamente un estado de deterioro cognitivo leve (Mild Cognitive Impairment (MCI), en inglés).

Un deterioro cognitivo leve se caracteriza por la pérdida de memoria, no obstante, no está estrechamente vinculado a que la persona vaya a desarrollar una demencia. La persona puede efectuar sus tareas cotidianas de manera competente e independiente [5].

El Alzheimer es la demencia más común entre personas mayores de 65 años acaparando entre un 60-80% de todos los casos. La enfermedad de Alzheimer ataca directamente a las neuronas del cerebro ocasionando una pérdida paulatina de las habilidades del paciente para llevar a cabo las actividades diarias de manera autónoma debido a la alteración de las funciones mentales como la memoria, el lenguaje, el razonamiento, la toma de decisiones, la conducta y el pensamiento [2].

Según las cifras oficiales publicadas por la Alzheimer's Disease International (ADI) en el año 2020, la enfermedad de Alzheimer afecta a 55 millones de personas en todo el mundo, siendo España, con 800 mil personas afectadas, el tercer país con mayor prevalencia de la enfermedad por detrás de Francia e Italia. Cada año se registran más de 10 millones de nuevos casos a nivel mundial, es decir, un nuevo caso cada 3 segundos. Si continúa con esta tendencia negativa, la cifra podría superar los 139 millones de personas afectadas para el año 2050 [6].

El Alzheimer, como tal, no causa la muerte sino las propias complicaciones pueden originar una neumonía aspirativa, una deshidratación, una desnutrición una infección urinaria, entre otras [7]. Entre los años 2000 y 2017, las muertes provocadas por el Alzheimer han aumentado en un 145%, situándose entre las 10 patologías más mortales del mundo [8]. Por esta razón, la medicina centra su principal foco de atención en encontrar tratamientos que prevengan o retrasen el avance de la enfermedad.

2.1 Causas del Alzheimer

En el cerebro de una persona diagnosticada con Alzheimer se produce un gran daño neuronal debido a una acumulación de placas seniles de proteína beta-amiloide y de ovillos neurofibrilares de proteína Tau que destruyen las neuronas del cerebro [2]. La muerte de las neuronas disminuye los niveles de producción de una sustancia química vital para el funcionamiento correcto del cerebro. Esta sustancia, denominada acetilcolina, es un neurotransmisor que permite a las neuronas estar en constante comunicación y está implicada en actividades mentales vinculadas al aprendizaje, la memoria, la conducta y el pensamiento [9].

Cuando se produce un daño neuronal en una determinada región del cerebro, las neuronas no pueden comunicarse entre sí y, en consecuencia, dicha región no puede desempeñar sus funciones con normalidad. Hoy en día, no se conoce una causa concreta a la que se le pueda atribuir el daño neuronal por esta acumulación anómala de las proteínas en el cerebro, sino que intervienen múltiples factores de riesgo que pueden aumentar la probabilidad de padecer la enfermedad de Alzheimer [10]:

- **Edad:** La incidencia de la enfermedad aumenta con la edad. La mayoría de los pacientes diagnosticados con Alzheimer superan los 60 años.
- **Influencia genética:** Aquellas personas cuya ascendencia presentaron algún caso de la enfermedad tienen una probabilidad de 2 a 4 veces mayor de padecerla en comparación a una persona sin un caso previo diagnosticado.
- **Género:** Las mujeres forman parte de un gran grupo de riesgo. Esto puede tener su explicación a su alta esperanza de vida en comparación a los hombres.
- **Conexión entre el Corazón y el Cerebro:** Un funcionamiento correcto del corazón está correlacionado con una buena salud del cerebro. Esta conexión tiene sentido ya que el corazón es el responsable de bombear y entregar la sangre al cerebro a través de los vasos sanguíneos.

- Nivel educativo: La escasa ejercitación cognitiva aumenta el riesgo de padecer la enfermedad.
- Otros factores de riesgo: Tabaco, Alcohol, Hipertensión arterial, Depresión, Diabetes y Obesidad..

2.2 Síntomas y Etapas de la Enfermedad

El primer síntoma que se manifiesta en la enfermedad de Alzheimer es la pérdida leve de la memoria. A medida que la enfermedad avanza progresivamente con el tiempo, las alteraciones de la memoria empeoran y se manifiestan otros síntomas que interfieren en las tareas cotidianas [2].

Dependiendo del estado en que se encuentre el paciente, se dan una serie de síntomas [11]:

➤ Estado Leve (Etapa 1)

En su etapa inicial, los síntomas de la enfermedad de Alzheimer no son perceptibles tanto por el paciente como por el entorno que le rodea:

- Lentitud en el habla.
- Pérdida leve de la memoria.
- Dificultad en la expresión y el aprendizaje.
- Alteraciones del comportamiento como irritabilidad, agresividad, decaimiento, apatía, cambios de humor, entre otras.

➤ Estado Moderada (Etapa 2)

En esta etapa, los síntomas de la enfermedad se agravan y empiezan a ser visibles por su entorno:

- Olvida hechos recientes.
- Alteración en la capacidad de razonamiento y comprensión.
- Incapacidad para recordar una información concreta.
- Incapacidad de gestionar sus finanzas.
- Problemas de atención y orientación.

- Dificultad para reconocer rostros.

➤ Estado Grave (Etapa 3)

En la etapa final, la enfermedad deja completamente incompetente al paciente:

- Presenta complicaciones para hablar, tragar y caminar.
- Incapacidad para reconocer a alguien.
- Desorientación constante.
- Pérdida del control corporal. Los pacientes más graves acaban encamados y necesitando una atención constante por una tercera persona.

2.3 Tratamientos y Prevención

Hoy en día, no existe un tratamiento que actúe directamente sobre la enfermedad de Alzheimer sino, más bien, hay ciertos tratamientos que hacen paliar sus síntomas y, en consecuencia, mejoran la calidad de vida del paciente.

Estos tratamientos actúan con una mayor eficacia en las primeras etapas de la enfermedad para ayudar a proteger al cerebro de sufrir más daño y, por lo tanto, un diagnóstico precoz y una intervención temprana es de gran importancia [9]. Más aún, un diagnóstico precoz de la enfermedad permite realizar estudios clínicos en la fase leve de la enfermedad para crear nuevos tratamientos con mayor probabilidad de éxito a fin de ralentizar o revertir el avance de la enfermedad de Alzheimer.

Capítulo 3. Machine Learning

En 1952 nace el término de la Inteligencia Artificial gracias al informático John McCarthy [12], pero no es hasta finales del siglo XX cuando esta tecnología empieza a coger impulso y llega a su máxima esplendor en estos últimos años fruto del gran avance en investigación y desarrollo en el campo de la informática.

La Inteligencia Artificial es una subdisciplina del campo de la informática que consiste en crear sistemas o máquinas que puedan imitar capacidades relativamente sencillas del ser humano como, por ejemplo: reconocer voces, analizar patrones, conducir, tomar decisiones, resolver problemas, caminar, etc [13].

Es importante remarcar que imitar no significa que dicha capacidad sea una capacidad cognitiva; es posible programar un brazo robótico inteligente para realizar un movimiento específico, pero no tiene la capacidad de aprender a realizar el movimiento por sí mismo. En este punto es donde entra en juego el Machine Learning. El Machine Learning o Aprendizaje Automático es una rama de la Inteligencia Artificial que estudia cómo crear sistemas o máquinas que puedan aprender por sí solas mediante una serie de algoritmos capaces de predecir y generalizar comportamientos a partir de una información suministrada en forma de ejemplos. Esta tecnología tiene la capacidad para identificar patrones y tomar decisiones a partir de una gran cantidad de información que son imposibles de procesar y manejar por el ser humano [14].

Dentro del aprendizaje automático, hay tres grandes categorías:

➤ Aprendizaje Supervisado

Son aquellos en los que existe una variable explícita o “etiqueta” a predecir por cada conjunto de características de los datos de entrada (Figura 3.1).

Datos de Entrada	Características									Etiqueta	
	fixed acidity	volatile acidity	citric acid	chlorides	total sulfur dioxide	density	pH	sulphates	alcohol	quality	color
	5.2	0.34	0.0	0.05	63.0	0.991	3.68	0.79	14.0	6	red
	6.2	0.55	0.45	0.049	186.0	0.997	3.17	0.5	9.3	6	white
	7.15	0.17	0.24	0.119	178.0	0.995	3.15	0.44	10.2	6	white
	6.7	0.64	0.23	0.08	119.0	0.995	3.36	0.7	10.9	5	red
	7.6	0.23	0.34	0.043	129.0	0.993	3.12	0.7	10.4	5	white
	5.7	0.22	0.2	0.044	113.0	0.998	3.22	0.46	8.9	6	white
	7.1	0.47	0.29	0.024	142.0	0.995	3.12	0.48	12.0	8	white
	9.7	0.31	0.47	0.062	33.0	0.998	3.27	0.66	10.0	6	red
	7.6	0.21	0.44	0.036	119.0	0.991	3.01	0.7	12.8	6	white
	5.8	0.32	0.28	0.032	115.0	0.989	3.16	0.57	13.0	8	white

Figura 3.1. Base de Datos de la Calidad de Vino.

Los algoritmos de ML aprenden a predecir dicha etiqueta bajo un proceso de entrenamiento en el cual se le enseña un histórico pasado, del cual ya conocemos el valor de la etiqueta a predecir.

El aprendizaje supervisado se aplica en:

- Problemas de clasificación: Cuando la variable a predecir se trata de una categoría/clase. Por ejemplo: predecir el color del vino según los valores de su composición química.
- Problemas de regresión: Cuando la variable a predecir se trata de un valor numérico. Por ejemplo: predecir el valor de una acción en bolsa en un determinado momento.
- Aprendizaje no Supervisado

Son aquellos en los que no existe una variable a predecir como tal. No hay una “etiqueta” que se corresponda a una entrada de datos determinada y, por lo tanto, sólo podemos describir la estructura que siguen los datos para obtener un mayor conocimiento sobre ellos [14].

El aprendizaje no supervisado se aplica en:

- Problemas de clustering: A través de grupos de observaciones podemos percibir gráficamente qué relación existe entre las características de los datos de entrada.
- Reducción de dimensionalidad: Elimina las características de los datos de entrada que no aportan ninguna información significativa mientras que, conserva las características que si lo hacen.

➤ Aprendizaje Reforzado

No existen unas etiquetas predefinidas que indiquen el valor de salida esperado sino el sistema interactúa con el entorno, ejecuta una acción y recibe una recompensa acorde al logro hacia su objetivo. El sistema aprende por su propia experiencia a ejecutar la mejor acción, en determinadas situaciones, que le haga aproximarse a su objetivo.

El aprendizaje reforzado se aplica en sistemas de conducción autónoma, chat Bot, la robótica, los videojuegos, etc.

Actualmente, la aplicación del ML está presente en los principales sectores como la computación, las finanzas, la medicina, el servicio de atención al cliente, la industria y en los juegos.

3.1 Deep Learning

El Deep Learning es un campo específico del ML que intenta emular el funcionamiento del cerebro humano a través de un sistema de red neuronal artificial que analiza los datos. A partir del análisis, es posible obtener patrones de comportamiento de los datos y así anticiparse a los problemas [15].

Este campo sigue el mismo principio del ML, la única diferencia que existe es que los algoritmos Deep Learning no requieren de la intervención humana para identificar los factores de variación o características más importantes que puedan explicar los datos de tipo no estructurados (audio, imágenes, texto, etc...). Son los propios algoritmos quienes se encargan de extraer las características más representativas de los datos [16].

Los algoritmos Deep Learning más conocidos son la Red Neuronal Convolucional y la Red Neuronal Recurrente.

3.1.1 Red Neuronal Artificial

La red neuronal artificial o también conocida como perceptrón multicapa es la base de las redes neuronales convolucionales y recurrentes. Es una red inspirada en el comportamiento biológico de las neuronas del ser humano, que son capaces de aprender y generalizar comportamientos adquiridos en ejemplos anteriores a nuevos ejemplos que nunca habían visto [17].

Del mismo modo que nuestro cerebro está constituido por neuronas interconectadas entre sí, una red neuronal artificial (Figura 3.2) está formada por neuronas artificiales conectadas entre sí y agrupadas en diferentes niveles que denominamos capa de entrada, capas ocultas y capa de salida.

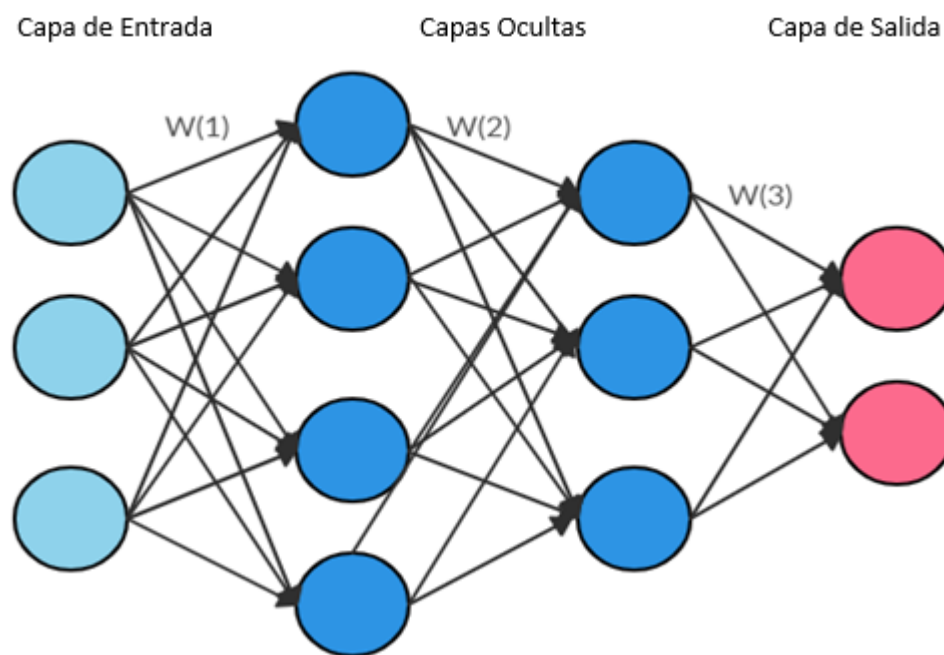


Figura 3.2. Estructura de la RNA.

En una red neuronal artificial, los valores de la capa de entrada como los de salida son conocidas, en cambio, los valores de entrada y salida de las capas ocultas son desconocidas. Por esta razón, reciben el nombre de capas ocultas.

3.1.1.1 Algoritmo Forward Propagation

Durante el entrenamiento de una red neuronal se hace uso del algoritmo Forward Propagation (Figura 3.3) para conseguir que los valores de las características de los datos que entran por las neuronas de la capa de entrada se propaguen por cada una de las neuronas que forman las capas ocultas, mediante operaciones de multiplicaciones y sumas, hasta llegar a la capa de salida dónde se elabora la salida o predicción de la red [17].

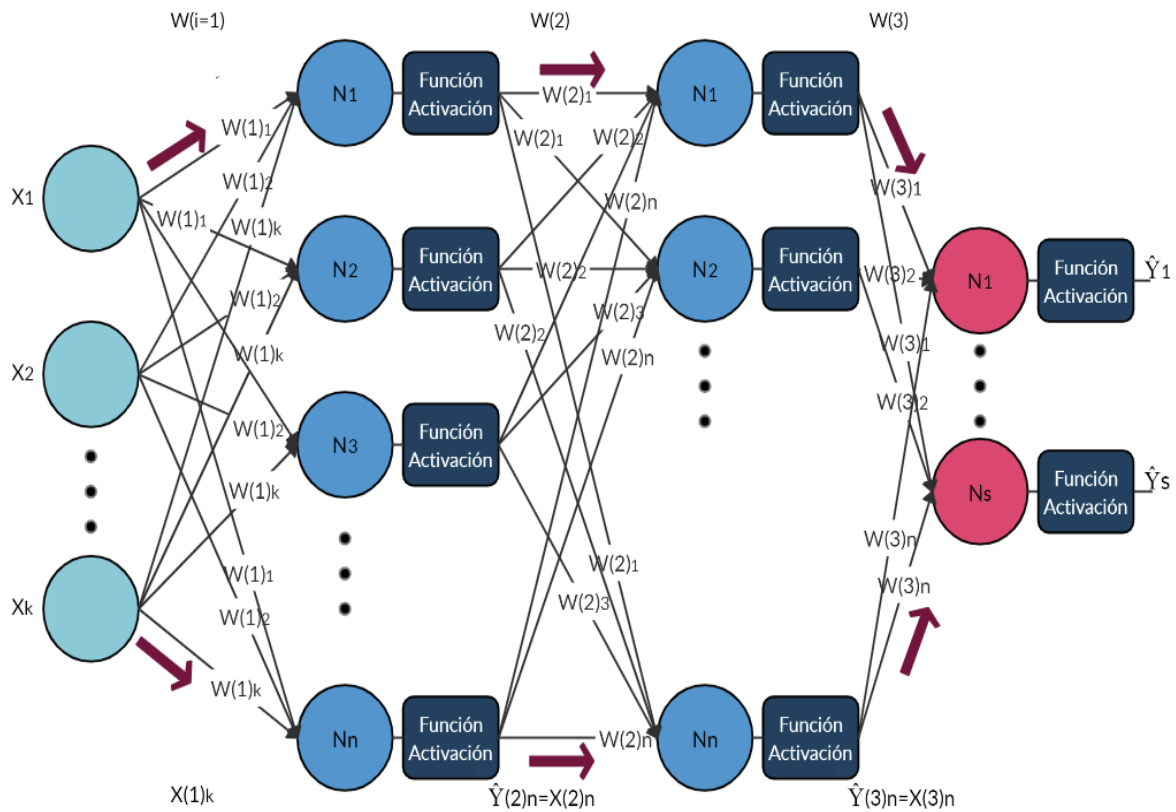


Figura 3.3. Algoritmo Forward Propagation.

Cada una de las conexiones entre las neuronas lleva asociado un peso o parámetro W , que la red neuronal debe aprender cómo debe ajustarlos en cada iteración de entrenamiento para generar una salida o predicción que se aproxime lo más posible al valor de la etiqueta predefinida para cada ejemplo del conjunto de datos de entrada. (Este proceso iterativo de ajuste de los parámetros de la red

se encargará el algoritmo BackPropagation, que más adelante entraremos en detalle).

El valor de salida de una neurona de la capa oculta se obtiene aplicando una función de activación al resultado de la combinación de una suma ponderada de todas las entradas de la capa anterior.

$$\hat{Y}_{(i)_n} = F(\sum_1^n w_n^{(i-1)} \times X_{(i-1)_n}) \quad (3.1)$$

Por tanto, siguiendo en la misma línea, la salida de la red ($\hat{Y}$) es el producto de aplicar una función de activación al resultado obtenido por la combinación de una suma ponderada de todos los valores de salida de la última capa oculta.

$$\hat{Y}_s = F(\sum_1^n w_n^{(i)} \times X_{(i)_n}) \quad (3.2)$$

Esta salida generada por la red tendrá tantos valores como el número de neuronas haya en la capa de salida.

3.1.1.2 Función de Activación

La función de activación es una función matemática encargada de permitir el paso de la información de una capa a otra de la red y, además, agrega no linealidad a la propia red para poder resolver así problemas mucho más complejos. A continuación, en la Figura 3.4 se puede apreciar la diferencia entre un modelo lineal y un modelo no lineal para clasificar dos clases.

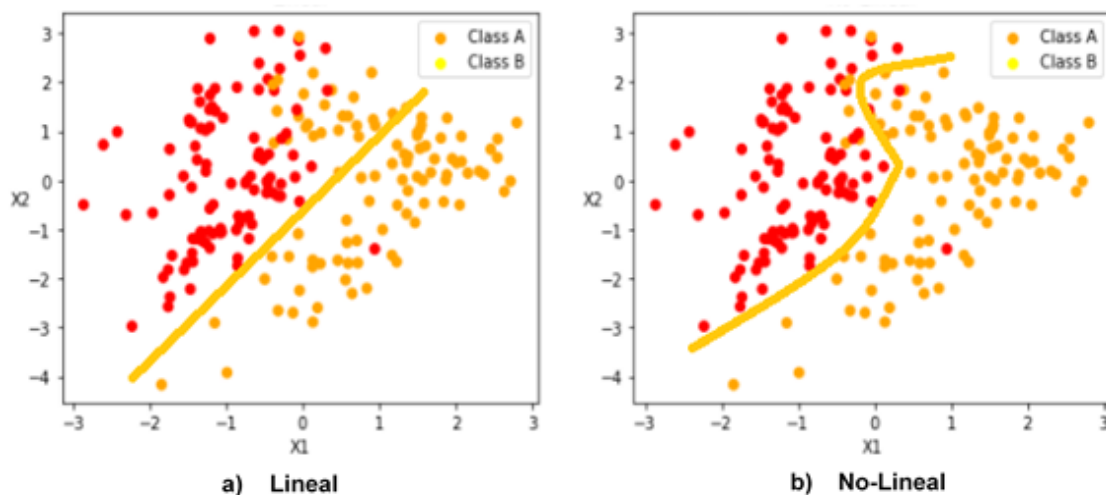


Figura 3.4. Comparación de un Modelo Lineal vs No-Lineal.

Las funciones de activación más conocidas son [18]:

- Función Sigmoidal (Figura 3.5):

El rango de valores a la salida oscila entre 0 y 1.

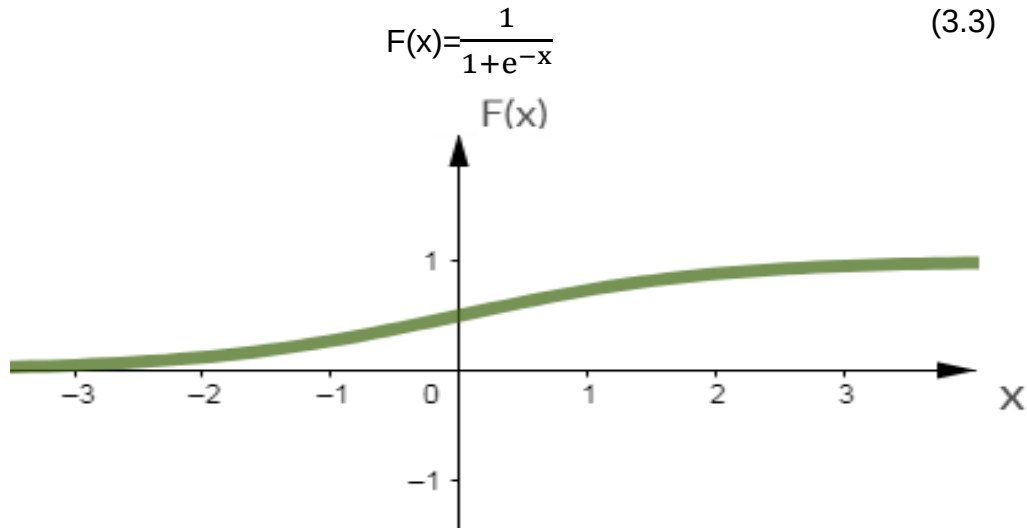


Figura 3.5. Función de Activación Sigmoidal.

Una de las principales desventajas de trabajar con la función Sigmoidal en las capas ocultas es la presencia de saturación para grandes valores de entrada. Esta situación provoca que los valores a la salida siempre tiendan a ser 0 o 1 y, por tanto, no se puedan apreciar muchos cambios durante el entrenamiento de la red. Hay que tener en cuenta que los algoritmos de optimización del aprendizaje precisan de cambios significativos para calcular el mínimo de la función de pérdida. Cuando la función se satura, los cambios son mínimos y el proceso de entrenamiento puede tomar mucho más tiempo e incluso puede ser que no converja al mínimo que estamos buscando.

La función Sigmoidal se suele utilizar en la capa de salida de la red neuronal para clasificar dos categorías/clases.

- Función Tangente Hiperbólica (Figura. 3.6):

El rango de valores a la salida oscila entre -1 y 1.

$$F(x) = \frac{\sinh x}{\cosh x} \quad (3.4)$$

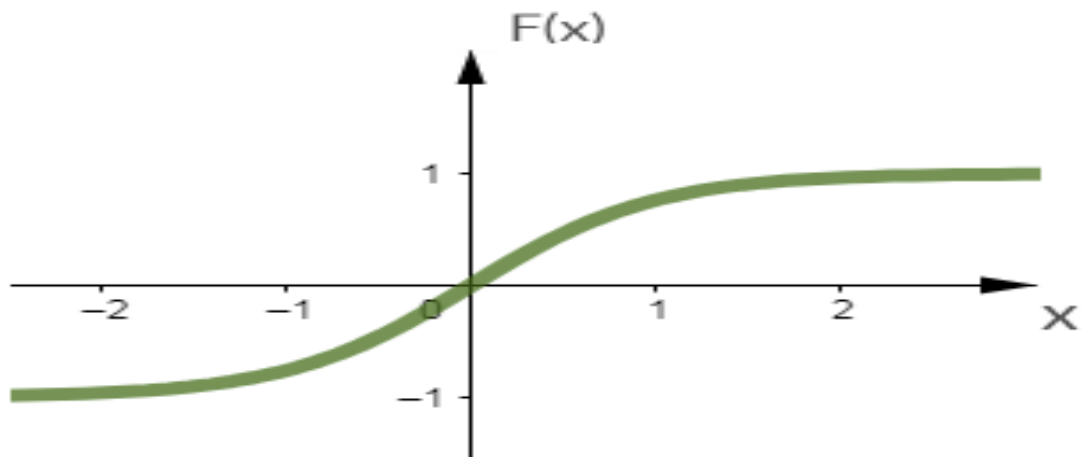


Figura 3.6. Función de Activación Tangente Hiperbólica.

Así como la función Sigmoidal, esta función presenta los mismos problemas para converger y llegar al mínimo absoluto.

- Función ReLU (Figura. 3.7):

El rango de valores a la salida oscila entre 0 e infinito.

$$F(x) = \max(0, x) \quad (3.5)$$

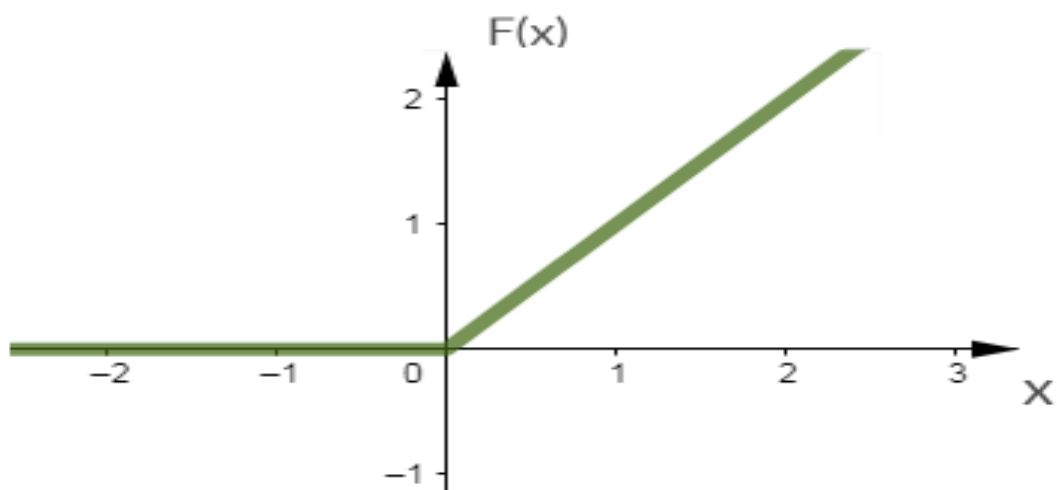


Figura 3.7. Función de Activación ReLU.

Si bien el problema de saturación existente en las funciones anteriores desaparece, la red no puede aprender para valores de entrada negativos dado que los valores a la salida no varían.

Este tipo de función de activación se sugiere aplicar en las capas ocultas porque ofrece una convergencia más rápida del algoritmo de optimización.

- Función Softmax:

Actúa como normalizador y convierte el valor numérico de salida de una neurona en un valor probabilístico.

$$F(y)_j = \frac{e^{y_j}}{\sum_{k=1}^K e^{y_k}} \quad (3.6)$$

La función Softmax se aplica en la capa de salida para la clasificación de dos o más categorías/clases.

3.1.1.3 Función de Pérdida

Cualquier modificación de los valores de los pesos W de la red neuronal genera una predicción totalmente diferente ($\hat{Y}$).

La función de pérdida o error mide la diferencia entre la salida predicha por la red ($\hat{Y}$) y la salida esperada (Y_{real}). De hecho, permite cuantificar el error cometido para cada una de las combinaciones posibles de los pesos W de la red [17].

$$E(W) = \hat{Y}(w) - Y \quad (3.7)$$

A través de los algoritmos de optimización del aprendizaje se busca minimizar esta función para mejorar la capacidad de aprendizaje de la red. Cuanto más cercano a cero sea la función de error, entonces mejor será el modelo predictivo.

En función del objetivo de nuestro problema, la función de error puede ser de regresión como, por ejemplo, el Error Cuadrático Medio, o de clasificación como, por ejemplo, el: Error de Entropía Cruzada Categórica.

3.1.1.4 Algoritmo BackPropagation

El objetivo del algoritmo BackPropagation consiste en modificar los valores de los pesos de tal manera que la red pueda aprender cómo minimizar la función de error en cada iteración de entrenamiento.

Para ello, el algoritmo (Figura 3.8) calcula el gradiente de la función de error para entender cómo se propaga el error por cada una de las neuronas de la red, empezando por las neuronas de la capa de salida hasta llegar a las neuronas de la capa de entrada.

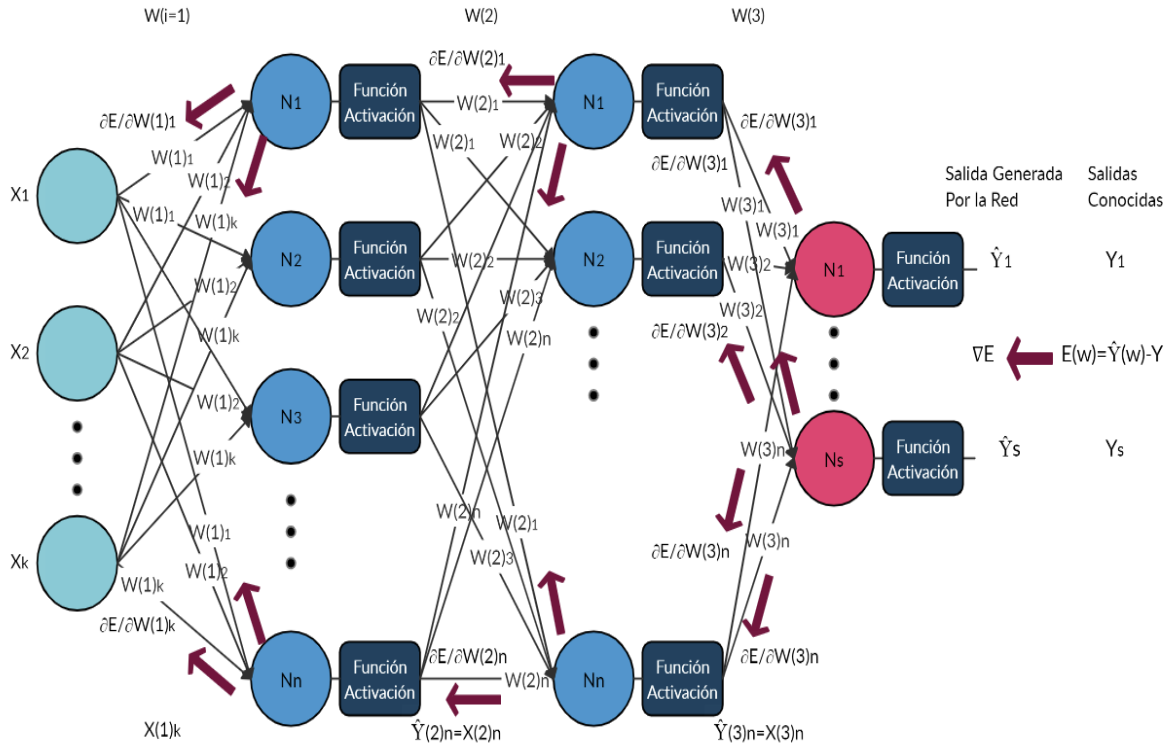


Figura 3.8. Algoritmo BackPropagation.

Cada una de las neuronas recibe un error que es proporcional a su contribución sobre la función de error y viene expresado como la derivada parcial de la función de error respecto al peso asociado en dicha conexión.

Una vez conocido cuál es el impacto de cada neurona en la función de error, el algoritmo emplea un algoritmo de optimización para encontrar qué valores de los pesos W permiten reducir el error a la salida de cada neurona, así como el error a la salida de la red en la siguiente iteración de entrenamiento [17]

No hay que confundir las iteraciones con las épocas de entrenamiento. Las iteraciones es el número de veces que el algoritmo de optimización se aplica en una época para minimizar la función de error mientras que, las épocas es el número de veces que se enseña el conjunto de datos al algoritmo. Ambas variables son hiperparámetros definidas para el algoritmo de optimización.

3.1.1.5 Algoritmo del Gradiente Descendente

El Gradiente Descendente es la base de aprendizaje de muchas técnicas de Machine Learning. Es un proceso de optimización de aprendizaje automático que consiste en encontrar el conjunto de valores de los pesos W de la red que minimizan la función de error en cada iteración de entrenamiento, ayudando así a mejorar la capacidad de aprendizaje y obtener predicciones más precisas [17].

En la Figura 3.9 se puede visualizar cómo el algoritmo comienza en un punto aleatorio de la función de error y desciende por su pendiente hasta llegar al punto más bajo de la función. Por mera simplicidad se ha representado la función de error en un plano 2D para describir el funcionamiento del algoritmo.

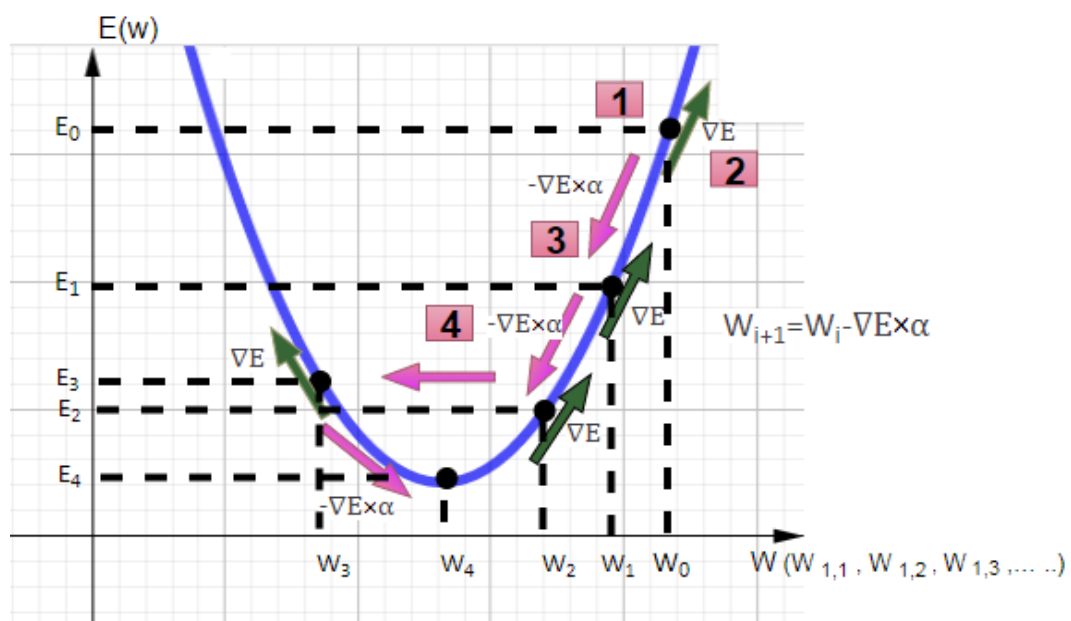


Figura 3.9. Funcionamiento del Algoritmo del Gradiente Descendente.

El proceso de aprendizaje por los que atraviesa la red neuronal hasta lograr minimizar la función de error son los siguientes:

1. Al inicio del entrenamiento, los pesos o parámetros W de la red están inicializados a un valor aleatorio. En ese mismo estado, el algoritmo Forward Propagation genera una predicción por cada ejemplo del conjunto de entrenamiento y las compara con las salidas reales. Se usa esa diferencia como una estimación estadística de la función de error en todo el conjunto.

2. Por medio del algoritmo Backpropagation se calcula el gradiente de la función de error para dichos pesos. El gradiente en ese punto nos informa la dirección en que la función de error aumenta.
3. El algoritmo del Gradiente Descendiente actualiza los pesos de la red tomando la dirección opuesta al gradiente con un paso α . Con este nuevo conjunto de pesos actualizados, el algoritmo Forward Propagation vuelve a generar una predicción por cada ejemplo del conjunto de entrenamiento y calcula una nueva estimación estadística de la función de error en todo el conjunto.

$$W_{i+1} = W_i - \nabla E \times \alpha \quad (3.8)$$

La tasa de aprendizaje α , es un valor que permanece fijo durante todo el entrenamiento e indica el paso tomado en cada actualización de los pesos. A veces, la elección de un valor adecuado para la tasa de aprendizaje resulta ser una tarea muy complicada. Un aumento de la tasa de aprendizaje puede conducir a una convergencia más rápida del algoritmo, pero también puede dar lugar a oscilaciones en torno a un mínimo local sin poder alcanzarlo. Por el otro lado, una tasa de aprendizaje demasiado pequeña hace que el algoritmo tenga una convergencia estable hacia un mínimo, aunque el tiempo necesario para alcanzarlo puede llegar a ser muy alto.

4. Por cada iteración de entrenamiento, se repite el procedimiento de los dos puntos anteriores hasta encontrar el mínimo global de la función de error.

En el algoritmo del gradiente descendiente se entiende por una iteración de entrenamiento cuando la red neuronal ha evaluado la función de error en todo el conjunto de entrenamiento. Por tanto, actualiza los pesos una sola vez por cada época.

3.1.1.6 Algoritmos de Optimización del Aprendizaje

La elección del algoritmo de optimización para su modelo de red neuronal puede significar la diferencia entre buenos resultados en minutos, horas o días.

Algunos de los optimizadores más populares son [17]:

- Descenso estocástico del gradiente o SGD (por las siglas en inglés de Stochastic Gradient Descent):

Es una variación del gradiente descendiente. El SGD actualiza los pesos W de la red por cada ejemplo elegido al azar del conjunto de entrenamiento.

Una actualización continua brinda una idea inmediata del rendimiento de un modelo, pero también es mucho más costoso desde el punto de vista computacional.

- Adam:

Es uno de los algoritmos de optimización de descenso más potente y rápido.

Adam ofrece una mejor propuesta que SGD. Durante el entrenamiento, el SGD mantiene una tasa de aprendizaje única para todas las actualizaciones de los pesos, en cambio, Adam asigna una tasa de aprendizaje a cada peso que se va adaptando a medida que disminuye la función de error.

Cuando no hay suficientes datos o el modelo de la red neuronal es muy complejo, los algoritmos de optimización con tasa de aprendizaje adaptativa obtienen mejores resultados y, además, una ventaja de este tipo de algoritmo de optimización es que no es necesario ajustar la tasa de aprendizaje.

- Algoritmo de gradiente adaptativo o Adagrad (por las siglas en inglés de Adaptive Gradient Descent):

Adapta la tasa de aprendizaje de los pesos en proporción a su historial de actualizaciones. Asigna una baja tasa de aprendizaje a los pesos más actualizados y una alta tasa de aprendizaje para aquellos pesos con pocas actualizaciones.

La principal debilidad de Adagrad es que para los pesos más actualizados llega un punto en que la tasa de aprendizaje se vuelve tan infinitamente pequeña que el algoritmo ya no es capaz de aprender.

➤ Adadelta:

Es una extensión de Adagrad que busca reducir su agresiva disminución de la tasa de aprendizaje.

3.1.2 Red Neuronal Convolucional

El sentido de la vista es uno de los campos donde más se ha invertido en investigación. Su órgano receptor es el globo ocular (ojo), que es capaz de identificar objetos dentro de su campo de visión, percibir su profundidad, distinguir perfectamente sus contornos y separarlos de sus fondos. La Red Neuronal Convolucional o CNN (por las siglas en inglés de Convolutional Neural Network) es un tipo de red neuronal artificial que imita la estructura de la corteza visual de los animales, dado que es el sistema de procesamiento visual más potente, siendo capaz de identificar y extraer las características más importantes que pueden explicar los datos observados [19].

La implementación de la CNN se lleva a cabo en entornos dónde se busca clasificar imágenes 2D y 3D, reconocer objetos en escena, identificar rostros, entre otros.

3.1.2.1 Arquitectura de la Red Neuronal Convolucional

La arquitectura de una CNN (Figura 3.10) está compuesta por tres tipos de capas, cada una de las cuales aporta una funcionalidad específica a la red [20]:

➤ Capa Convolucional

El objetivo principal de la capa convolucional es la extracción de características o rasgos visuales en las imágenes.

La capa convolucional tiene un número de filtros o kernels, cada uno de los cuáles aprenden a extraer diferentes características de la imagen. Un filtro o kernel es una matriz con unos parámetros o pesos aleatorios que la CNN intenta aprender cómo ajustarlos en cada iteración para extraer de manera correcta las características de la imagen.

Cuando la capa convolucional recibe como entrada una imagen, esta aplica sobre ella un número de filtros generando nuevas imágenes con diferentes características espaciales de la imagen original (mapas de características).

Una red neuronal convolucional está caracterizada por tener varias capas convolucionales con un nivel de abstracción cada vez más complejo, es decir, una primera capa puede aprender o detectar elementos básicos (aristas, líneas, bordes, etc....), una segunda capa puede aprender patrones compuestos de los elementos básicos de la capa anterior y, así sucesivamente. Las capas convolucionales más profundas irán aprendiendo patrones o formas más complejas como puede ser un rostro, la silueta de un animal, distinción entre objetos y más.

➤ Capa de Agrupación

Esta capa se implementa después de la capa convolucional para resaltar las características más representativas de los mapas de características y, a su vez, reduce la cantidad de parámetros que se deben ajustar durante el entrenamiento.

La capa de agrupación máxima o también conocida como Maxpooling es la más utilizada para implementar una CNN.

➤ Capa Conectada

Las capas conectadas son las capas tradicionales de una red neuronal artificial y se emplean al final de la CNN para realizar la clasificación:

- Capa Plana o flatten: Transforma la salida multidimensional de la capa convolucional o de agrupación a una matriz unidimensional.
- Capa Totalmente Conectada o Densa: Recibe como entrada las características extraídas por las capas convolucionales y de agrupaciones, e intenta aprender ajustando sus pesos para elaborar la predicción a través de la capa clasificadora.
- Capa Clasificadora o Capa de Salida: Actúa como clasificador. Se agrega al final de la red con el número de neuronas como clases/categorías a predecir.

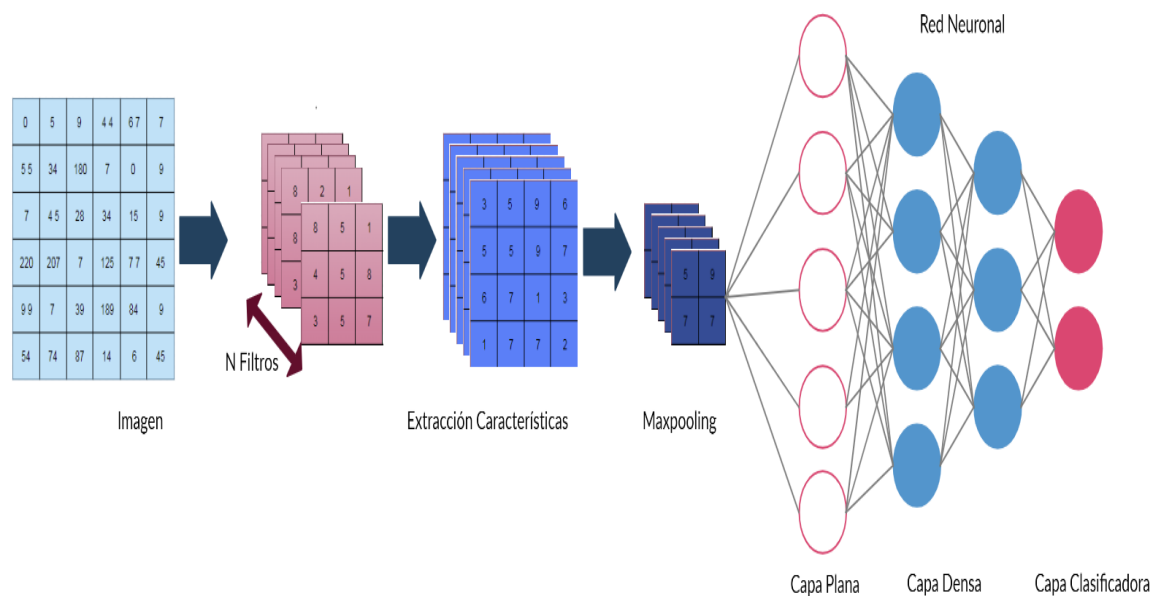


Figura 3.10. Ejemplo de Aplicación de una CNN.

3.1.3 Red Neuronal Recurrente

La CNN es idónea para detectar patrones visuales, sin embargo, no está capacitada para relacionar datos de una secuencia. Una secuencia es una serie de datos (palabras, imágenes, sonidos) que siguen un orden específico y tienen únicamente significado cuando se analizan en conjunto. A veces, la secuencia de datos y la dinámica temporal que conecta los datos aportan más información que simplemente el contenido de cada dato de manera individual. La Red Neuronal Recurrente o RNN (por las siglas en inglés de Recurrent Neural Network) es la tecnología más poderosa dentro del aprendizaje supervisado permitiendo analizar datos con una naturaleza secuencial (video, temperatura, clima, etc.). [21].

Una red neuronal recurrente (Figura 3.11) se puede ver como una secuencia de múltiples copias de una red neuronal artificial con dos entradas: el dato actual y la activación del estado oculto anterior, es decir, incorpora una retroalimentación con objeto de crear memoria a la red y permitir de este modo generar la predicción o salida no sólo usando la muestra actual sino también la salida generada en las muestras anteriores. Por otro lado, las redes vistas anteriormente (RNA y CNN) sólo pueden generar la predicción a partir de la entrada actual porque no son

capaces de recordar acerca de lo que ha sucedido en el pasado, excepto lo que hay reflejado en sus pesos tras el entrenamiento.

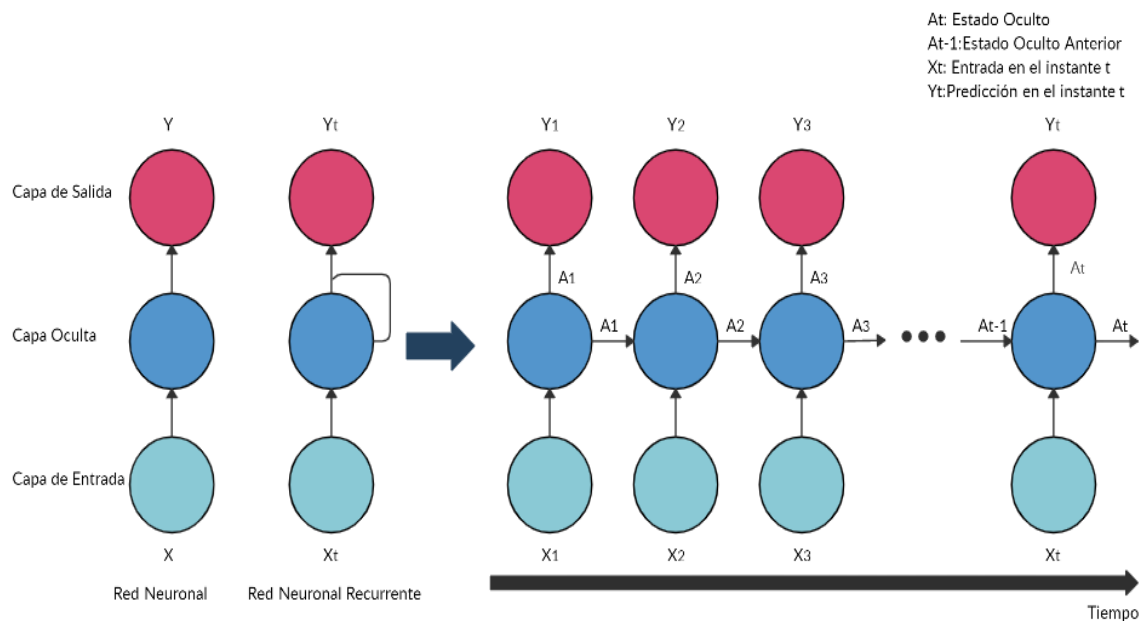


Figura 3.11. Estructura de la RNN.

El principal problema de la RNN es la desaparición del gradiente, este fenómeno provoca que la información entre estados temporales muy distantes se pierda. La secuencia procesada por la RNN debe ser relativamente corta para que las activaciones anteriores del estado oculto tengan un efecto relevante en la predicción.

Para solventar este problema de las dependencias temporales a largo plazo surgen un tipo especial de redes neuronales recurrentes llamadas LSTM y GRU.

➤ Red recurrente LSTM

Las LSTM pueden aprender dependencias tanto a corto como a largo plazo. Son capaces de recordar un dato relevante de la secuencia y preservarlo por varios instantes de tiempo [22]

A diferencia de una red neuronal recurrente convencional, la celda LSTM (Figura 3.12) incorpora una entrada y una salida adicional. Este elemento adicional se le conoce como celda de estado y es la memoria de la red LSTM.

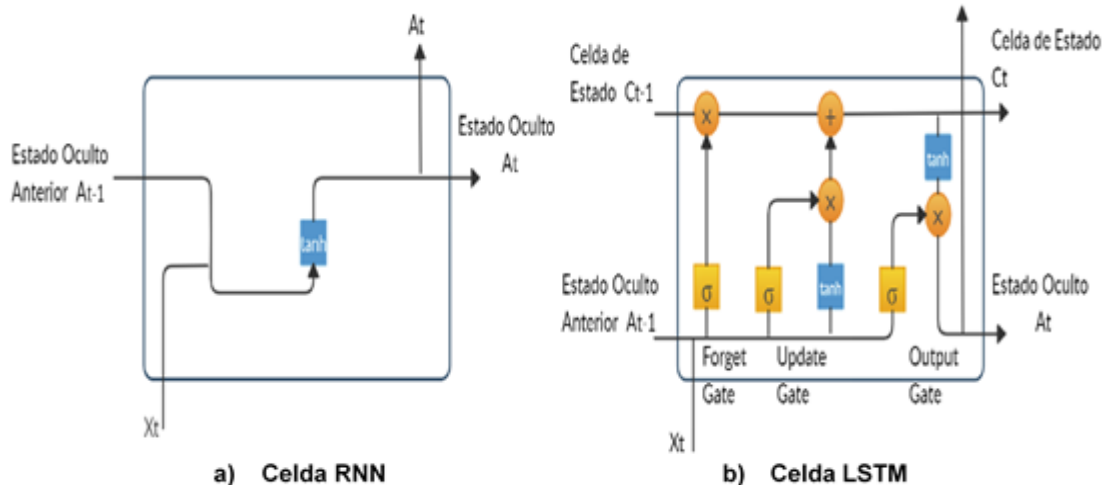


Figura 3.12. Comparación de una celda RNN vs LSTM.

Con esta celda de estado, la red LSTM puede controlar la memoria de la red usando una serie de compuertas que discriminan entre la información relevante e irrelevante que pasará a la salida:

- Forget gate: Elimina información de la memoria.
- Update Gate: Actualiza o añade nuevos elementos a la memoria.
- Output Gate: Crea el estado oculto actualizado.

A través del entrenamiento, la red LSTM trata de aprender a reconocer qué información es la relevante para la predicción.

El funcionamiento de la red es muy similar a como el lóbulo temporal del cerebro humano, que es el encargado de la memoria a largo plazo, analiza una secuencia. Mantiene la información de la secuencia que considera relevante y desecha el resto de la información.

➤ Red recurrente GRU

Es computacionalmente más eficiente que LSTM y, además, ofrece un mejor rendimiento en pequeños conjuntos de datos [22].

La red GRU (Figura 3.13) utiliza la puerta de actualización (update gate) para agregar la información que es relevante para la predicción mientras que, la puerta de reinicio (reset gate) desecha la que no es.

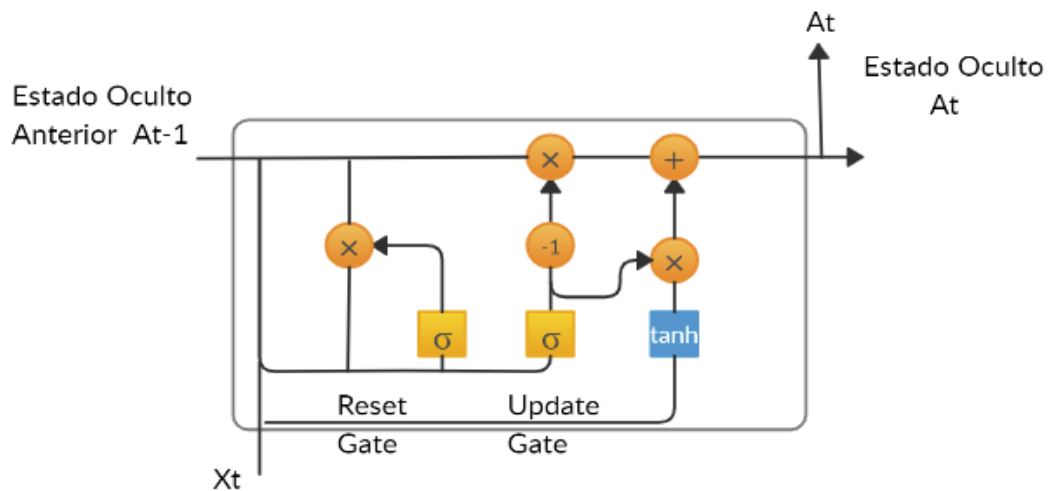


Figura 3.13. Celda GRU.

3.1.3.1 Arquitectura y Aplicación

La arquitectura de una CNN está diseñada para que los datos de entrada y salida siempre tengan el mismo tamaño y, por lo tanto, no pueden procesar secuencia de datos como, por ejemplo, un texto que se caracteriza por tener una cantidad variable de palabras con diferentes longitudes. Por otro lado, una red neuronal recurrente es capaz de procesar secuencia de datos tanto en la entrada como a la salida sin importar su tamaño y, además, conservar la correlación existente entre los diferentes elementos de una secuencia.

En función de su aplicación de diseño, la arquitectura de una red neuronal recurrente puede ser [21]:

- One to Many (Figura 3.14).

La red neuronal recurrente recibe un único dato como entrada y la salida responde con una secuencia de datos.

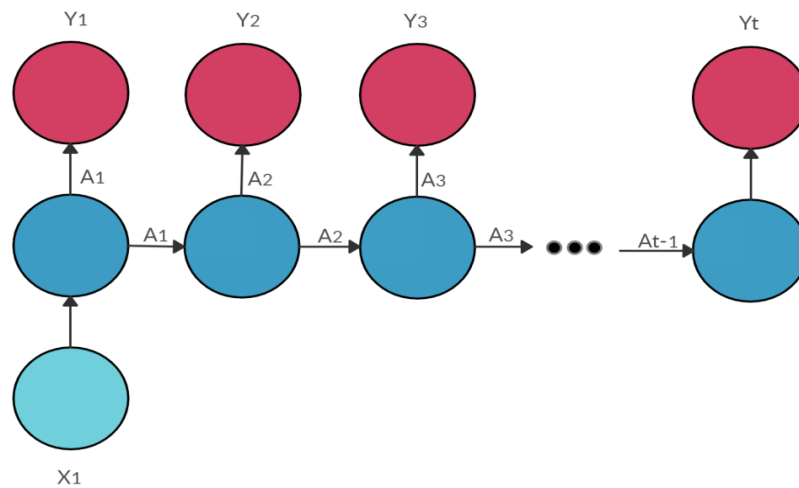


Figura 3.14. Arquitectura One to Many.

Un ejemplo de aplicación de la arquitectura One to Many es la descripción de una imagen, donde la entrada corresponde a una imagen y la salida es un texto que describe el contenido de dicha imagen.

➤ Many to One (Figura 3.15).

La red neuronal recurrente recibe una secuencia de datos como entrada y la salida responde con una categoría/clase.

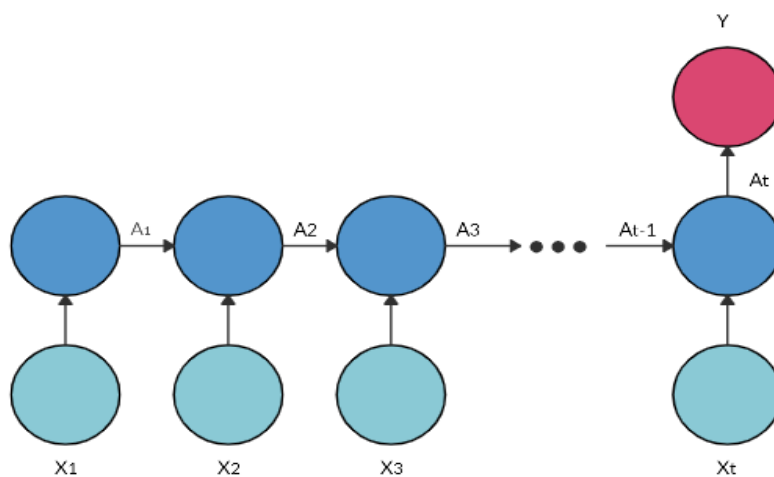


Figura 3.15. Arquitectura Many to One.

Un ejemplo de aplicación de la arquitectura Many to One es el análisis de sentimientos, donde la entrada corresponde a un texto y la salida es una categoría que indica el estado de ánimo.

Otras aplicaciones: Clasificación de videos, Detección de movimiento.

➤ Many to Many (Figura 3.16).

La red neuronal recurrente recibe una secuencia de datos como entrada y la salida responde con una secuencia desfasada un paso de tiempo.

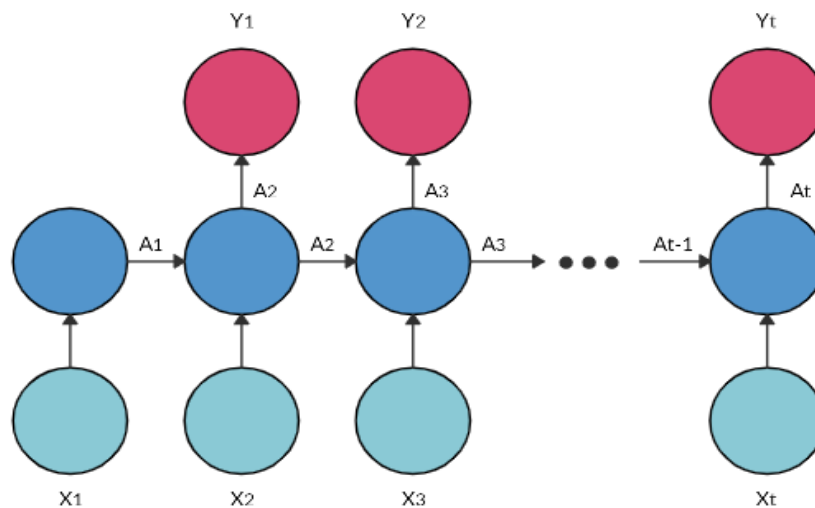


Figura 3.16. Arquitectura Many to Many.

Un ejemplo de esta aplicación de la arquitectura Many to Many es el traductor automático, donde la entrada corresponde a un texto escrito en un idioma y la salida es un texto traducido al idioma solicitado.

Otras aplicaciones: Reconocimiento de la escritura a mano, Reconocimiento de la voz.

3.1.4 Librerías

Cualquier persona con unos conocimientos básicos en el aprendizaje automático puede implementar un modelo Deep Learning de forma sencilla gracias a dos librerías de código abierto que se complementan: Tensorflow y Keras [23].

- Tensorflow es una librería de computación matemática que ejecuta de forma rápida y eficiente los gráficos de flujo de datos. Un gráfico de flujo está formado por operaciones matemáticas representadas sobre nodos, cuya entrada y salida es un vector multidimensional de datos (tensor).

Tensorflow fue desarrollada por Google Brain para sus aplicaciones de aprendizaje automático profundo. Es el motor que ejecuta y entrena la red neuronal.

- Keras es una librería desarrollada por Google que proporciona una sintaxis de alto nivel escrita en Python y una interfaz sencilla para la creación de redes neuronales.

Además de Keras y Tensorflow, hay otras librerías de código abierto que reducen la complejidad de la sintaxis del código y ayudan al programador a diseñar un modelo Deep Learning. Cabe mencionar algunas de ellas como Matplotlib, Nibabel, Skimage, Sklearn, Numpy, entre muchas otras.

3.2 Diagnóstico de la Habilidad de un modelo de ML

El objetivo de un modelo ML es aprender patrones de comportamiento de un conjunto de datos y generalizarlos a nuevos datos que el modelo nunca ha visto.

Una forma de medir la capacidad de generalización de un modelo consiste en dividir el conjunto de datos en dos conjuntos (Figura 3.17): conjunto de entrenamiento y conjunto de prueba. El modelo se entrena con el conjunto de entrenamiento para aprender a detectar patrones de estos y, posteriormente, evalúa su capacidad de generalización o predictiva en el conjunto de prueba. Normalmente, esta división del conjunto de datos suele ser un 80% para el conjunto de entrenamiento y un 20% para el conjunto de prueba.

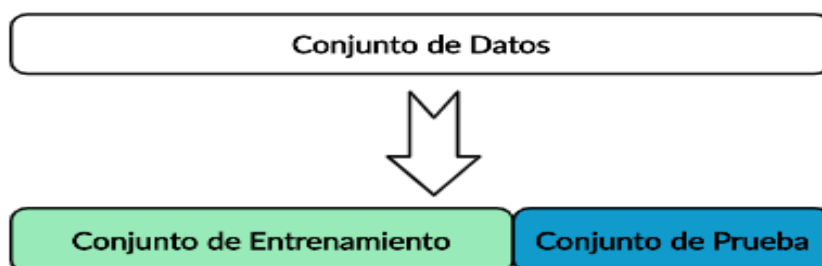


Figura 3.17. Split Train-Test.

En el ML es relativamente sencillo configurar un modelo, aunque conseguir minimizar el error de varianza y, al mismo tiempo, el error de sesgo del modelo es una tarea bastante complicada. Estas dos medidas de error están estrechamente relacionadas con la capacidad del modelo [24]:

➤ Modelo con Sub-Ajuste (Figura 3.18)

Si un modelo es demasiado simple tenderá alcanzar un escenario de Sub-Ajuste.

Un modelo con Sub-Ajuste se caracteriza por un alto sesgo y una baja varianza. Esto quiere decir que el modelo no es capaz de detectar patrones en los datos del conjunto de entrenamiento ni predecir correctamente los datos del conjunto de prueba, pero la capacidad de generalización del modelo no dependerá del conjunto de datos con el cual se ha entrenado.

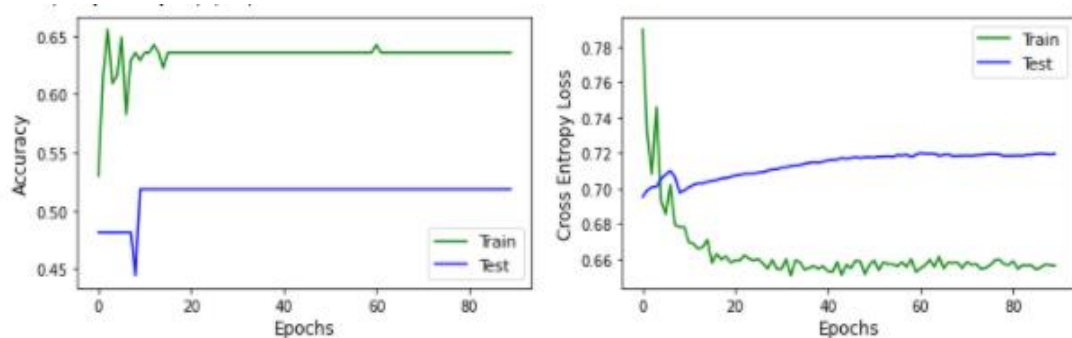


Figura 3.18. Curva de Aprendizaje de un Modelo Deep Learning con Sub-Ajuste.

➤ Modelo con Buen-Ajuste (Figura 3.19)

Encuentra el equilibrio entre una capacidad de aprendizaje muy simple o demasiada compleja con el fin de conseguir un bajo error de sesgo y varianza. Un modelo con buen ajuste aprende a detectar patrones correctamente de los datos del conjunto de entrenamiento y generaliza con acierto en los datos del conjunto de prueba.

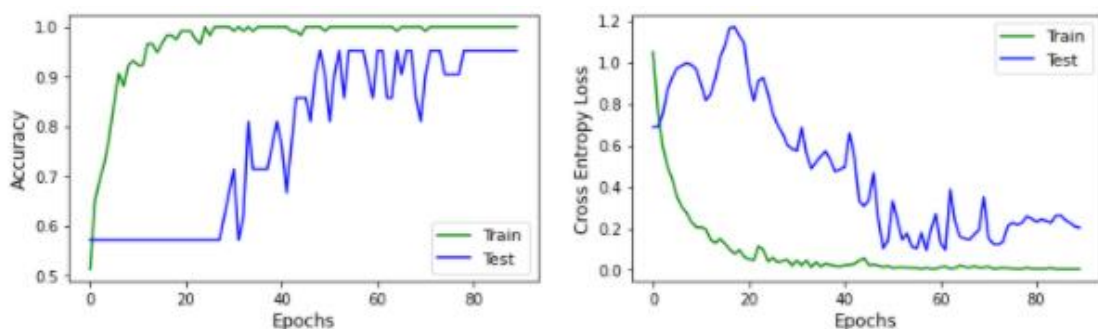


Figura 3.19. Curva de Aprendizaje de un Modelo Deep Learning con Buen Ajuste.

➤ Modelo con Sobreajuste (Figura 3.20)

Si un modelo dispone de pocos datos para su aprendizaje y, además, resulta ser muy complejo, es decir, contiene muchos parámetros (pesos) que ajustar durante el entrenamiento entonces el modelo tenderá a sobre-ajustarse con demasiada facilidad.

Un modelo con Sobre-Ajuste es el problema más común dentro del aprendizaje automático aplicado. Se caracteriza por un bajo sesgo y una alta varianza, es decir, el modelo tiene un buen desempeño para aprender a detectar patrones en los datos del conjunto de entrenamiento, pero no es capaz de generalizar correctamente con los datos del conjunto de prueba. El modelo aprende demasiado bien el conjunto de datos de entrenamiento llegando ajustarse a unas características muy específicas de los datos de entrenamiento que no tienen relación causal con la función objetivo y estará generalizando correctamente sólo aquellos datos idénticos a los de nuestro conjunto de entrenamiento.

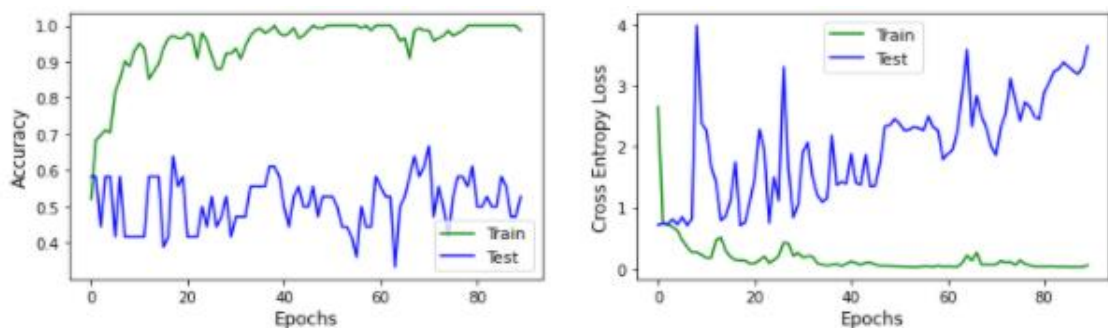


Figura 3.20. Curva de Aprendizaje de un Modelo Deep Learning con Sobreajuste.

Por suerte, es posible combatir la aparición o, al menos, reducir el impacto del sobreajuste de dos maneras:

- Mayor cantidad de datos: Un modelo entrenado con más datos tiende a generalizar mejor.

En situaciones donde no sea posible recopilar más datos se aplican las técnicas de regularización.

➤ Técnicas de Regularización:

- **Tamaño del modelo:** Reducir la complejidad del modelo disminuye considerablemente el impacto del sobre-ajuste. Simplemente con reducir el número de capas o la cantidad de neuronas podemos hacer que nuestro modelo sea más simple y, por tanto, son menos parámetros que deben ajustarse durante el entrenamiento

- **Regularizadores de peso:** Cuando las redes neuronales sufren de sobre-ajuste, sus pesos tienden a ser muy elevados. Un modelo con unos pesos elevados es un claro signo de una red más compleja.

Los regularizadores de peso se encargan de penalizar esos pesos muy elevados y los mantiene pequeños durante el aprendizaje.

- **Capa de Abandono o Deserción:** Un porcentaje de neuronas seleccionadas al azar de una capa son apagadas durante el entrenamiento de la red y son las otras quien deben de intervenir y manejar la representación requerida para realizar las predicciones de las neuronas apagadas.

De este modo, todas las neuronas participan en el aprendizaje y la red no se adapta demasiado a los ejemplos del entrenamiento.

- **BatchNormalization:** A medida que los datos fluyen a través de las capas de una red neuronal profunda, la normalización en los datos de entrada se va perdiendo y esto provoca que la capacidad de aprendizaje del modelo vaya empeorando.

El BatchNormalization es una técnica que consigue mantener los datos normalizados ofreciendo así una convergencia más rápida del algoritmo de optimización y evitando de esta forma tener que entrenar una red con muchas épocas con el objetivo de converger al mínimo de la función de error. Esta técnica, además de actuar como normalizador, interviene de algún modo en un efecto de regularización.

3.2.1 Métricas de Rendimiento

Las métricas de rendimiento se emplean en ML para medir la habilidad que tiene un modelo en predecir correctamente nuevos datos. Se puede ver como unos indicadores que reflejan una idea aproximada de cómo funcionará nuestro modelo una vez sea puesto en producción. Las métricas pueden ser de dos tipos, dependiendo del problema: Métricas de regresión y Métricas de clasificación. Para no entrar mucho en detalle en esta temática, sólo veremos las métricas de clasificación que van a ser aplicadas, en el capítulo 4, para medir las prestaciones del modelo de clasificación implementado [25]:

- Matriz de Confusión (Tabla 3.1): Describe con exactitud el rendimiento del modelo.

		Predicho	
Real	Clases	Negativo	Positivo
	Negativo	VN (Verdadero Negativo)	FP (Falso Positivo)
	Positivo	FN (Falso Negativo)	VP (Verdadero Positivo)

Tabla 3.1. Matriz de Confusión.

Para esta matriz de confusión de dos clases, la clase negativa podría corresponderse, por ejemplo, a personas con una buena salud mental mientras que, la clase positiva podría corresponderse a personas diagnosticadas con alguna demencia.

- Curva ROC (o curva característica de funcionamiento del receptor): Es un gráfico que interpreta la habilidad del modelo de clasificación para distinguir entre la clase positiva y la clase negativa. El gráfico (Figura 3.21) representa la tasa de falsos positivos (eje x) frente a la tasa de verdaderos positivos (eje y).

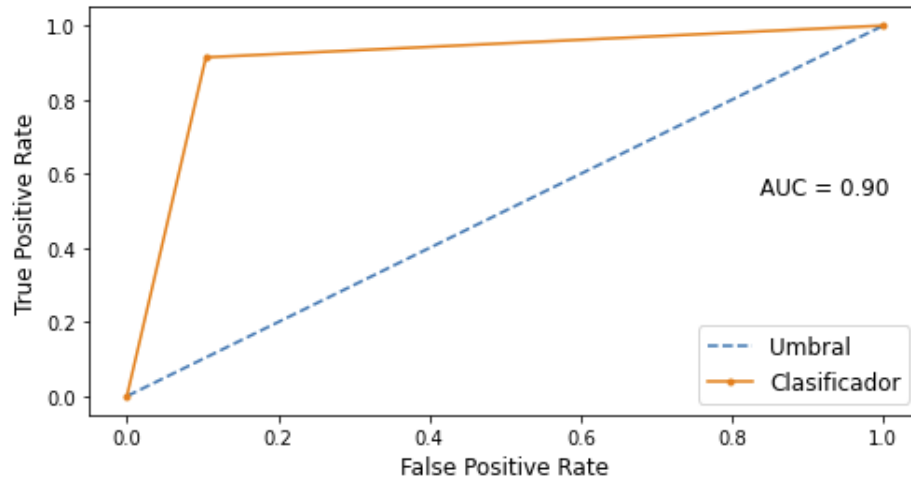


Figura 3.21. Curva ROC.

Un modelo con una habilidad perfecta se representa en el punto (0,1), la zona más alejada al umbral (línea azul).

- ROC AUC: Es una medida numérica del área de la Curva ROC. Cuanto más se aproxime a uno el ROC AUC, mejor será la habilidad del modelo para distinguir entre clase positiva y clase negativa.

Esta métrica al igual que las siguientes se mueven en un rango [0-1]. Un valor 0 indica un rendimiento pésimo del modelo mientras que, un valor 1 indica un rendimiento perfecto.

- Exactitud (Accuracy): Es la relación entre el número de predicciones correctas y el número total de predicciones realizadas.

$$Accuracy = \frac{VP + VN}{VP + FN + VN + FN} \quad (3.9)$$

En problemas de clasificación con clases desbalanceadas, la exactitud no es la métrica más apropiada para medir el rendimiento del modelo.

- Sensibilidad (Recall): Es la relación entre el número de predicciones positivas correctas y el número total de casos positivos.

$$Recall = \frac{VP}{VP + FN} \quad (3.10)$$

Indica una estimación de la capacidad del modelo para identificar correctamente un caso como positivo entre todos los casos positivos.

- Especificidad (Specificity): Es la relación entre el número de predicciones negativas correctas y el número total de casos negativos.

$$Especificidad = \frac{VN}{VN + FP} \quad (3.11)$$

Indica una estimación de la capacidad del modelo para identificar correctamente un caso como negativo entre todos los casos negativos.

3.3. La Reproducibilidad de los algoritmos de Machine Learning

La mayoría de los algoritmos de ML son estocásticos, es decir, requieren del uso de la aleatoriedad durante el aprendizaje como, por ejemplo: aleatoriedad en la recopilación de datos, en el orden de observación, en el algoritmo de optimización, en el remuestreo, en la inicialización de los pesos, etc [26].

El uso de la aleatoriedad juega un papel muy importante en el aprendizaje automático aplicado. Tal es así, que puede evitar a los algoritmos quedar atrapados en mínimos locales y obtener resultados que los algoritmos deterministas no pueden alcanzar como también ser la principal causa de que un mismo modelo de ML, previamente entrenado con un conjunto de datos, reproduzca diferentes predicciones en un conjunto de prueba. Esta situación provoca que el programador tenga describir el rendimiento de un modelo utilizando un resumen estadístico que mida el rendimiento medio tras varias ejecuciones de entrenamiento en lugar del rendimiento obtenido tras una sola ejecución.

Gracias a estos dos métodos, Control de la varianza y Control de la estabilidad, podemos obtener una estimación sólida del rendimiento de un modelo estocástico [27].

- Control de la Estabilidad

El uso de la aleatoriedad ofrece al modelo más flexibilidad a su aprendizaje, pero también puede hacer que el modelo sea más inestable. Esto quiere decir que un mismo modelo entrenado con el mismo conjunto de entrenamiento puede reproducir un rendimiento diferente en el mismo conjunto de prueba.

Este comportamiento inestable que presenta el modelo se puede controlar con el uso de ensambles, o bien entrenando el modelo N-veces para obtener una media del rendimiento.

- Control de la Varianza

Cuando un modelo dispone de pocos datos para ajustar una gran cantidad de parámetros (pesos), este se vuelve sensible a pequeñas fluctuaciones de los datos provocando que un mismo modelo entrenado con diferentes conjuntos de datos de entrenamiento reproduzca diferentes rendimientos en un mismo conjunto de prueba.

En conclusión, la habilidad predictiva de un modelo que presenta una alta varianza depende del conjunto de datos con el cual se ha entrenado. Sólo generalizará o predecirá con acierto aquellos datos idénticos a los de entrenamiento.

Para reducir el error de varianza de un modelo, existen dos técnicas que nos ayudan a ello:

- Validación Cruzada K-Fold.
- Ensamblés.

3.3.1 Validación Cruzada K-Fold

Es una de las técnicas más utilizadas para evaluar el rendimiento de un modelo de aprendizaje automático, dado que proporciona una estimación fiable y poco sesgada [28].

A través de la Validación Cruzada K-Fold podemos determinar que configuración específica de los hiperparámetros de un modelo consigue alcanzar la mejor capacidad de generalización o predictiva en nuevos datos. El procedimiento que sigue esta técnica (Figura.3.22) es el siguiente:

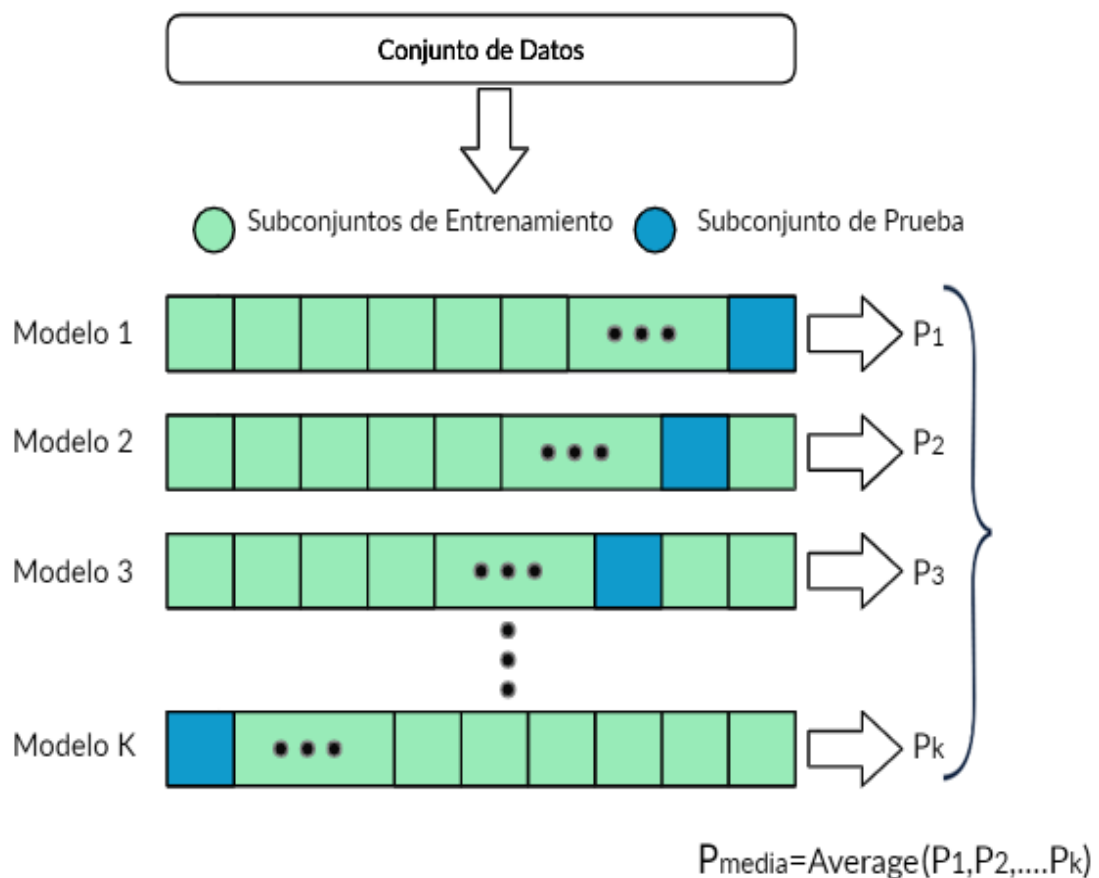


Figura 3.22. Técnica de la Validación Cruzada K-Fold.

1. En primer lugar, el conjunto de datos se mezcla aleatoriamente y se divide en K subconjuntos o pliegues. La elección del valor de K viene determinado por el tamaño del conjunto de datos; un buen valor de K es 3 o 5 para grandes conjuntos de datos y 10 o más para pequeños conjuntos de datos.
2. En segundo lugar, el modelo se entrena con los K-1 subconjuntos (conjunto de entrenamiento) para aprender a detectar patrones de estos y, posteriormente, evalúa su capacidad de generalización o predictiva en el subconjunto restante (conjunto de prueba), cuyos datos no han sido vistos por el modelo. El proceso se repite K-1 veces hasta que todos los subconjuntos tengan la oportunidad de ser el conjunto de prueba.
3. Por último, la Validación Cruzada K-Fold estima el rendimiento del modelo promediando las predicciones obtenidas en cada conjunto de prueba.

$$P_M = \text{Average}(P_1, P_2, P_3, \dots, P_k) \quad (3.12)$$

En problemas de clasificación con pequeños conjuntos de datos o conjuntos desbalanceados, una mejor alternativa a esta técnica es la Validación Estratificada K-Fold dado que establece un equilibrio de las clases en cada subconjunto [29]

3.3.2. Ensamblajes

El aprendizaje conjunto o ensamblado es una técnica del aprendizaje automático que combina un conjunto de modelos para crear un modelo más confiable y robusto. En ocasiones, esta técnica mejora la capacidad predictiva puesto que la predicción final no depende exclusivamente de un sólo modelo sino de un conjunto de modelos entrenados para contribuir a resolver un mismo problema [30].

La idea central del aprendizaje conjunto es reducir la varianza e inestabilidad de los algoritmos de Machine Learning. Dentro del aprendizaje conjunto, hay dos vertientes [31]:

- Técnicas simples de conjunto (Figura 3.23):
 - Votación por mayoría

Múltiples modelos son entrenados a partir de un conjunto de entrenamiento para generar una predicción ('voto') por cada ejemplo del conjunto de prueba. La predicción final será la clase que reciba más de la mitad de los votos.

$$Prediccion_{final} = \frac{Pred_1 + Pred_2 + \dots + Pred_N}{N} \quad (3.13)$$

La votación por mayoría es una de las maneras más simples de combinar las predicciones de múltiples modelos de clasificación.

- Promedio

La predicción final será la media de las predicciones obtenidas por cada modelo del conjunto en cada ejemplo del conjunto de prueba. Todos los modelos del conjunto contribuyen de igual manera en la predicción final independientemente qué tan bien se haya desempeñado.

$$Prediccion_{final} = Mode(Pred_1 + Pred_2 + \dots + Pred_N) \quad (3.14)$$

El promedio se utiliza para combinar predicciones de múltiples modelos de regresión.

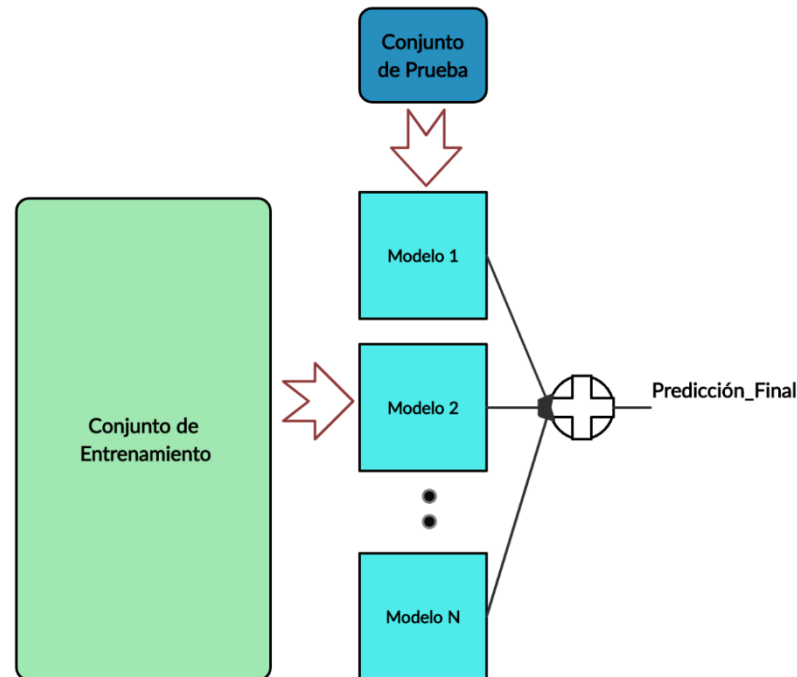


Figura 3.23. Técnicas Simples de Conjunto.

- Técnicas avanzadas de conjunto:
 - Bagging (Embolsado): Múltiples modelos entrenados con diferentes subconjuntos de datos combinan sus predicciones en un conjunto de prueba haciendo uso de las técnicas simples.
El embolsado es ideal para modelos que presenten síntomas de una alta varianza.
 - Boosting (Impulso): Múltiples modelos son entrenados para aprender a corregir los errores de predicción del modelo anterior. Al igual que el Bagging, las predicciones se combinan con las técnicas simples.

- **Stacking (Apilamiento):** Un nuevo modelo (meta-modelo) trata de aprender cómo combinar mejor las predicciones de múltiples modelos. La prioridad del Stacking es aumentar la fuerza predictiva del modelo.

Capítulo 4. Implementación del Clasificador

Una vez presentado los conceptos teóricos que son necesarios para comprender el diseño del modelo de ML implementado para clasificar la enfermedad de Alzheimer, se procede a describir, en los siguientes subapartados, cada uno de los pasos tomados en el diseño.

El diseño de un modelo de ML va mucho más allá que entrenar un modelo específico con un conjunto de datos de entrenamiento y evaluar, posteriormente, su capacidad predictiva en un conjunto de datos de prueba; es todo un proceso que incluye recopilar datos acordes al problema de estudio, probar diferentes arquitecturas de red, diferentes esquemas de procesamiento de los datos, diferentes configuraciones de los hiperparámetros del algoritmo de aprendizaje automático y todo ello con un único objetivo: maximizar el rendimiento del modelo predictivo para el área en el que se ha diseñado.

4.1. Conjunto de Datos

Todos los algoritmos de Machine Learning precisan de los datos para su aprendizaje. Si estos no generalizan ni representan el caso de estudio entonces repercutirá en una pésima capacidad predictiva del algoritmo.

Para el caso de estudio que abordamos en el proyecto, los datos vienen suministrados en forma de imágenes cerebrales PET obtenidos de diferentes grupos de sujetos.

4.1.1 Tomografía por Emisión de Positrones

La tomografía por emisión de positrones o PET (por las siglas en inglés de Positron Emission Tomography) es una técnica no invasiva de la medicina nuclear que permite obtener imágenes, en tres dimensiones (plano sagital, coronal y axial), de los órganos y tejidos del interior de nuestro organismo, y evaluar su funcionalidad a través de la detección de zonas de actividad metabólica a nivel celular [32]. En una imagen PET, la zona con más activación metabólica a nivel celular se ve reflejada con una mayor intensidad lumínica.

A diferencia de otras exploraciones como pueden ser la tomografía axial computarizada o la resonancia magnética, la PET puede detectar en las primeras etapas de una enfermedad cierta activación anormal en la zona afectada y no sólo cuando la estructura de la zona ha sido dañada a causa de la enfermedad. En el caso de la enfermedad de Alzheimer, la PET demuestra ser una herramienta efectiva para comprender los cambios metabólicos producidos en las neuronas durante el avance de la enfermedad [33].

A continuación, en la Figura 4.1 se ilustra una comparativa entre una imagen cerebral PET obtenida de un paciente con Alzheimer y un paciente con un cognitivo normal.

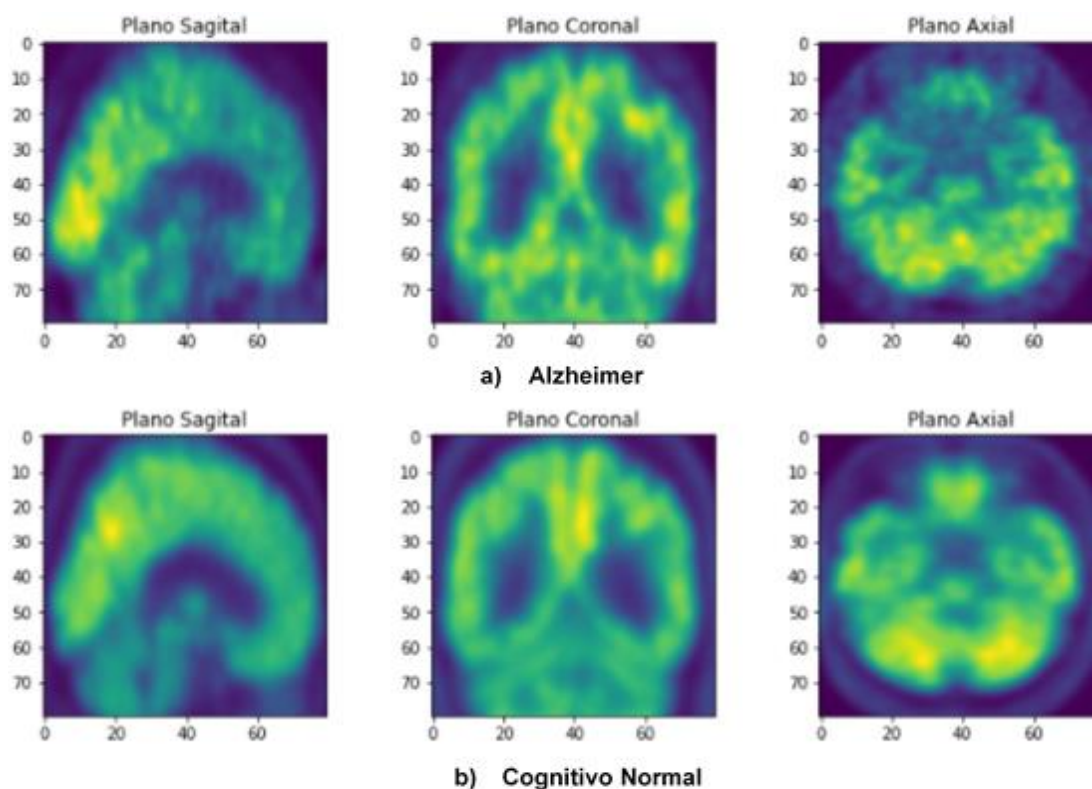


Figura 4.1. Cortes en los diferentes planos de una imagen cerebral PET de un paciente con Alzheimer y un paciente con un cognitivo normal.

Se puede observar como en el cerebro de un paciente con Alzheimer hay mucha menos activación metabólica de las neuronas, en comparación a un paciente con un cognitivo normal, debido al daño neuronal que ocasiona la propia enfermedad.

4.1.2 Base de Datos

Todas las imágenes cerebrales PET necesarias para el estudio han sido recopiladas de la base de datos de la Iniciativa de Neuroimagen de la Enfermedad de Alzheimer (ADNI), que está disponible públicamente en la página web (<http://adni.loni.usc.edu/>). Esta base de datos nace con la idea de poder realizar estudios, a futuro, que sean capaces de detectar la enfermedad de Alzheimer en las fases iniciales de su desarrollo, donde más efectivo van a ser los tratamientos.

Nuestro conjunto de datos recoge únicamente las imágenes cerebrales PET de la base de datos ADNI que se corresponden con sujetos diagnosticados con la enfermedad de Alzheimer, sujetos diagnosticados con un deterioro cognitivo leve

y sujetos diagnosticados con un cognitivo normal. La tabla siguiente (Tabla 4.1) muestra un resumen demográfico del conjunto de datos empleado:

Diagnóstico	Número	Edad	Sexo (M/F)
CN	68	75.84±4.93	43/25
MCI	111	76.39±6.96	76/35
AD	70	75.33±7.17	46/24

Tabla 4.1. Información Demográfica de las Imágenes Recopiladas

4.2. Arquitectura de Red Propuesta

Una elección equivocada en la arquitectura de red del modelo de clasificación puede limitar su capacidad de aprendizaje durante el entrenamiento. Por este motivo, es sumamente importante elegir una arquitectura que se adapte al conjunto de datos del problema a resolver.

En este caso, al tratarse de imágenes cerebrales PET 3D lo más lógico sería aplicar un modelo que siga una arquitectura CNN 3D dado que son idóneas para capturar con éxito las características de estas imágenes y, realizar las respectivas clasificaciones AD vs CN y MCI vs CN. Sin embargo, debido a las limitaciones del hardware no podemos trabajar directamente con este tipo de arquitectura para poder procesar y manejar toda la cantidad de información requerida en estas imágenes.

Por esta circunstancia, el proyecto propone un modelo de clasificación basado en una arquitectura de Red Neuronal Híbrida que recibe como datos de entrada secuencia de imágenes cerebrales PET 2D obtenidos a partir de la segmentación de una imagen cerebral PET 3D. De esta manera, el problema a nivel computacional que aparecía en la clasificación de imágenes cerebrales PET 3D se simplifica en una serie de problemas de clasificación de imágenes cerebrales PET 2D a lo largo de la dirección de un tercer plano, más concretamente, centramos el estudio en el plano Coronal, aunque puede extrapolarse a cualquier otro plano.

La arquitectura de la Red Neuronal Híbrida está formada por una Red Neuronal Convolutiva 2D (CNN 2D) conectada en cascada a una Red Neuronal Recurrente (RNN) (Figura 4.2).

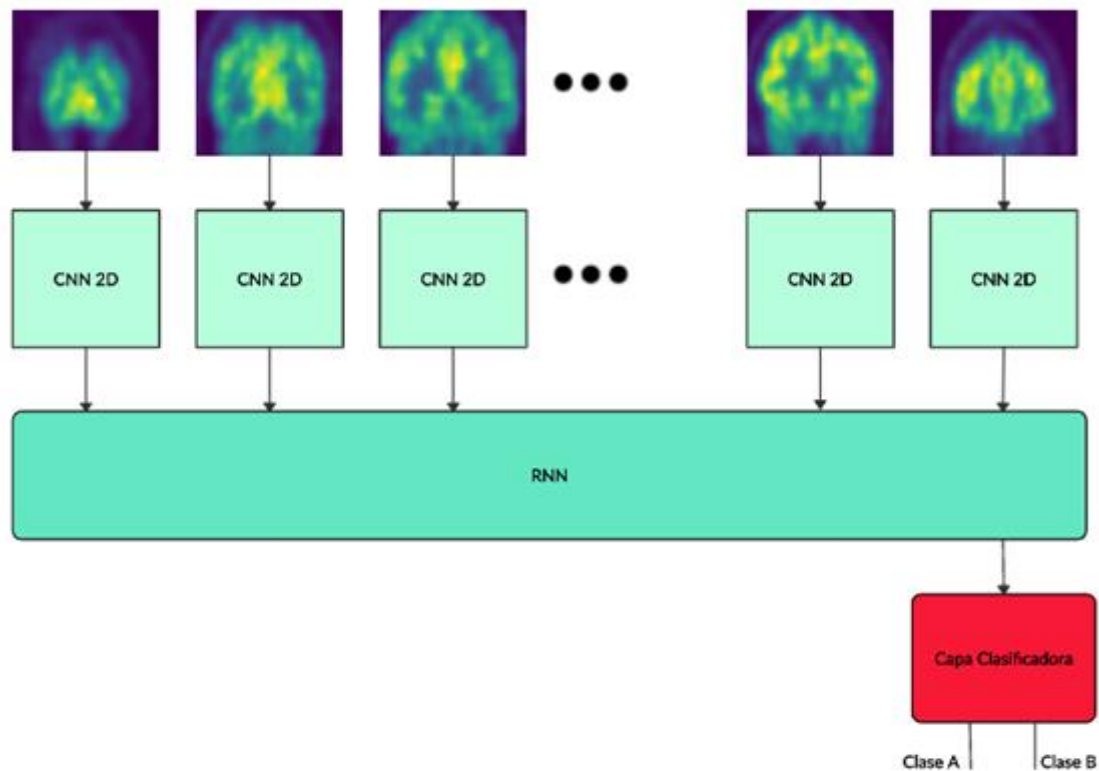


Figura 4.2. Diagrama de Flujo de la Arquitectura Propuesta.

La Red Neuronal Convolutiva 2D se encarga de aprender a capturar las características espaciales de cada corte (imagen PET 2D) de la secuencia mientras que, la Red Neuronal Recurrente, con una arquitectura Many to One, recibe las características espaciales capturadas por la CNN 2D y aprende a relacionar dichas características entre los diferentes cortes para elaborar la predicción a través de una capa clasificadora.

Las ventajas que ofrece este tipo de arquitectura, frente a otras, para analizar estas imágenes son las siguientes:

- Entrenar una CNN 2D requiere de mucho menos parámetros frente a una CNN 3D y, por lo tanto, es más fácil de entrenar para extraer las características espaciales.

- La incorporación de una red neuronal recurrente a la arquitectura permite analizar la secuencia de imágenes cerebrales PET 2D, siendo capaz de capturar características espaciales en la dirección perpendicular a las imágenes de la secuencia. Sin embargo, una red Lenet o Alexnet, que están basadas en una arquitectura CNN 2D, sólo pueden analizar el plano espacial de cada corte de la imagen cerebral PET 3D.
- Combinar una CNN 2D junto a una RNN nos permite obtener más información acerca de la imagen cerebral PET 3D que simplemente aplicando una red por separado. Por tanto, la habilidad del modelo para clasificar la enfermedad de Alzheimer mejora notablemente.

4.3. Procesamiento de las Imágenes cerebrales

Antes de modelar y procesar las imágenes cerebrales PET 3D, estas deben presentar un formato acorde a la arquitectura de red propuesta para nuestro modelo de clasificación.

Tras haber probado diferentes esquemas de procesamiento en las imágenes, se detalla, a continuación, paso por paso, el procesamiento con el cual se obtiene el mejor rendimiento de nuestro modelo de clasificación:

- Las imágenes cerebrales PET 3D que forman nuestro conjunto de datos presentan un formato NIfTI-1 y una dimensionalidad de 79×95×68 vóxeles (Un vóxel es la unidad cúbica que compone un objeto tridimensional, el equivalente al píxel en un objeto 2D). Con ayuda de la biblioteca NiBabel se carga en memoria cada una de estas imágenes y se le asigna una etiqueta (AD, MCI y CN) que define la clase/categoría a la que pertenece.

Posteriormente, cada imagen cerebral PET 3D se descompone en una secuencia que está formada por 18 imágenes cerebrales de 79×68 píxeles correspondientes a los cortes 10-14-18-22-26-30-34-38-42-46-50-54-58-62-66-70-74-78 del plano Coronal. Entre los cortes de la secuencia se establece un paso de separación para evitar así tener imágenes duplicadas que pueda provocar al modelo de clasificación aprender unas características muy específicas de la secuencia y, por lo tanto, no estaría

generalizando correctamente en imágenes cerebrales PET 3D de otros sujetos.

- Algunas imágenes de la secuencia presentan valores no numéricos (NaN). Con la intención de obtener unos datos lo más preparados posible de cara a mejorar el entrenamiento del modelo se reemplazan dichos valores NaN por un valor cero.
- El tamaño de las imágenes cerebrales de cada secuencia se reduce, por medio de la función `resize` de la biblioteca `Skimage`, a una dimensión de 50×50 píxeles (Figura 4.3). Con esta reducción de escala se sigue manteniendo la calidad de las imágenes que forman la secuencia y, además, permite al modelo ajustar mucho menos parámetros durante el entrenamiento para detectar patrones en las secuencias de imágenes cerebrales PET 2D.

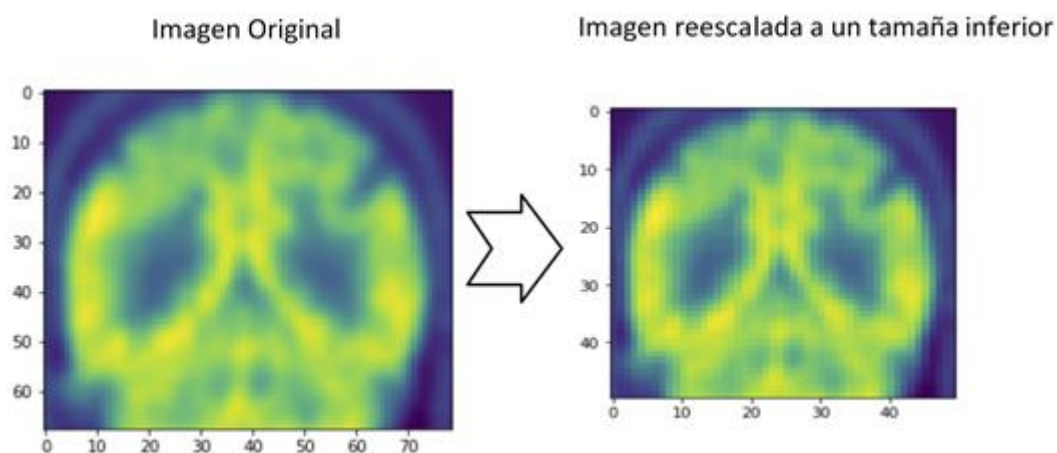


Figura 4.3. Redimensión de la imagen a un tamaño inferior.

- Al reducir la dimensión de las imágenes, los píxeles se comprimen en una cuadrícula más pequeña y, por tanto, sus valores de píxeles cambian. Para mantener todos los valores de píxeles de las imágenes cerebrales PET 2D en una misma escala de 0 a 1, se debe restar cada valor de píxel por el valor mínimo y se divide por el valor máximo. Este proceso, denominado normalización, garantiza que el algoritmo de optimización del aprendizaje pueda converger más rápido durante el entrenamiento.

Las etiquetas categóricas definidas, anteriormente, a cada imagen cerebral PET 3D se representa con un valor numérico (0 y 1) y mediante la

codificación One-Hot se convierte a un vector binario (Tabla 4.2). De este modo, es posible medir el error entre los valores predichos por el modelo ante una secuencia de imágenes que recibe como entrada y los valores reales asociados a dicha secuencia que ha sido obtenida a partir de la segmentación de una imagen PET 3D.

Clasificación	Categoría/Clase	Valor Numérico	Codificación One-Hot
AD vs CN	CN (Negativa)	0	01
	AD (Positiva)	1	10
MCI vs CN	CN (Negativa)	0	01
	MCI (Positiva)	1	10

Tabla 4.2. Aplicación de la Codificación One-Hot en las etiquetas.

4.4. Configuración del Modelo de Clasificación

Un modelo de clasificación que sigue una arquitectura híbrida posee una enorme cantidad de hiperparámetros que se deben tener en cuenta en el diseño como, por ejemplo: la elección de los cortes de la imagen cerebral PET 3D que formarán parte de la secuencia, la elección del número de capas de convolución, el número de filtros por cada capa convolucional, uso de la capa GRU o LSTM, el número de unidades por cada capa recurrente, la elección del algoritmo de optimización, etc.

El principal obstáculo al cual nos hemos enfrentado a la hora de entrenar nuestro modelo de clasificación y evaluar, respectivamente, su habilidad para clasificar la enfermedad en nuevos sujetos es la presencia de sobreajuste. El modelo clasifica correctamente las imágenes cerebrales del grupo de sujetos que se le han enseñado durante el entrenamiento, pero no es capaz de clasificar de la misma manera las imágenes cerebrales de un grupo de sujetos que nunca ha visto (grupo de sujetos de prueba). Este comportamiento se debe a la escasa cantidad de imágenes disponibles en comparación a la gran cantidad de parámetros que debe aprender a ajustar el modelo durante el entrenamiento.

Por medio de la técnica de la Validación Cruzada Estratificada 10-Fold se han entrenado y evaluado múltiples configuraciones de los hiperparámetros del modelo de clasificación hasta que se ha encontrado una configuración específica de este, el cual ha sido capaz de afrontar mejor los dos problemas de clasificación que abordamos en el proyecto (AD vs CN y MCI vs CN), pudiendo reducir al máximo el impacto del sobreajuste. La tabla de la Figura 4.4 y el esquema de la Figura 4.5 describen la configuración del modelo de clasificación con el que se ha experimentado el mejor rendimiento para clasificar la enfermedad de Alzheimer en 10 grupos de sujetos de prueba.

```
Model: "Modelo_Clasificación"
```

Layer (type)	Output Shape	Param
Conv2D_1 (TimeDistributed)	(None, 18, 46, 46, 8)	208
BatchNormalization_1 (Batch Normalization)	(None, 18, 46, 46, 8)	32
Conv2D_2 (TimeDistributed)	(None, 18, 42, 42, 16)	3216
BatchNormalization_2 (Batch Normalization)	(None, 18, 42, 42, 16)	64
MaxPooling_1 (TimeDistributed)	(None, 18, 21, 21, 16)	0
Conv2D_3 (TimeDistributed)	(None, 18, 17, 17, 32)	12832
BatchNormalization_3 (Batch Normalization)	(None, 18, 17, 17, 32)	128
MaxPooling_2 (TimeDistributed)	(None, 18, 8, 8, 32)	0
Dropout_1 (TimeDistributed)	(None, 18, 8, 8, 32)	0
Flatten (TimeDistributed)	(None, 18, 2048)	0
GRU (GRU)	(None, 32)	199872
BatchNormalization_4 (Batch Normalization)	(None, 32)	128
Dropout_2 (Dropout)	(None, 32)	0
Dense (Dense)	(None, 2)	66

```

Total params: 216,546
Trainable params: 216,370
Non-trainable params: 176

```

Figura 4.4. Arquitectura del Modelo de Clasificación.

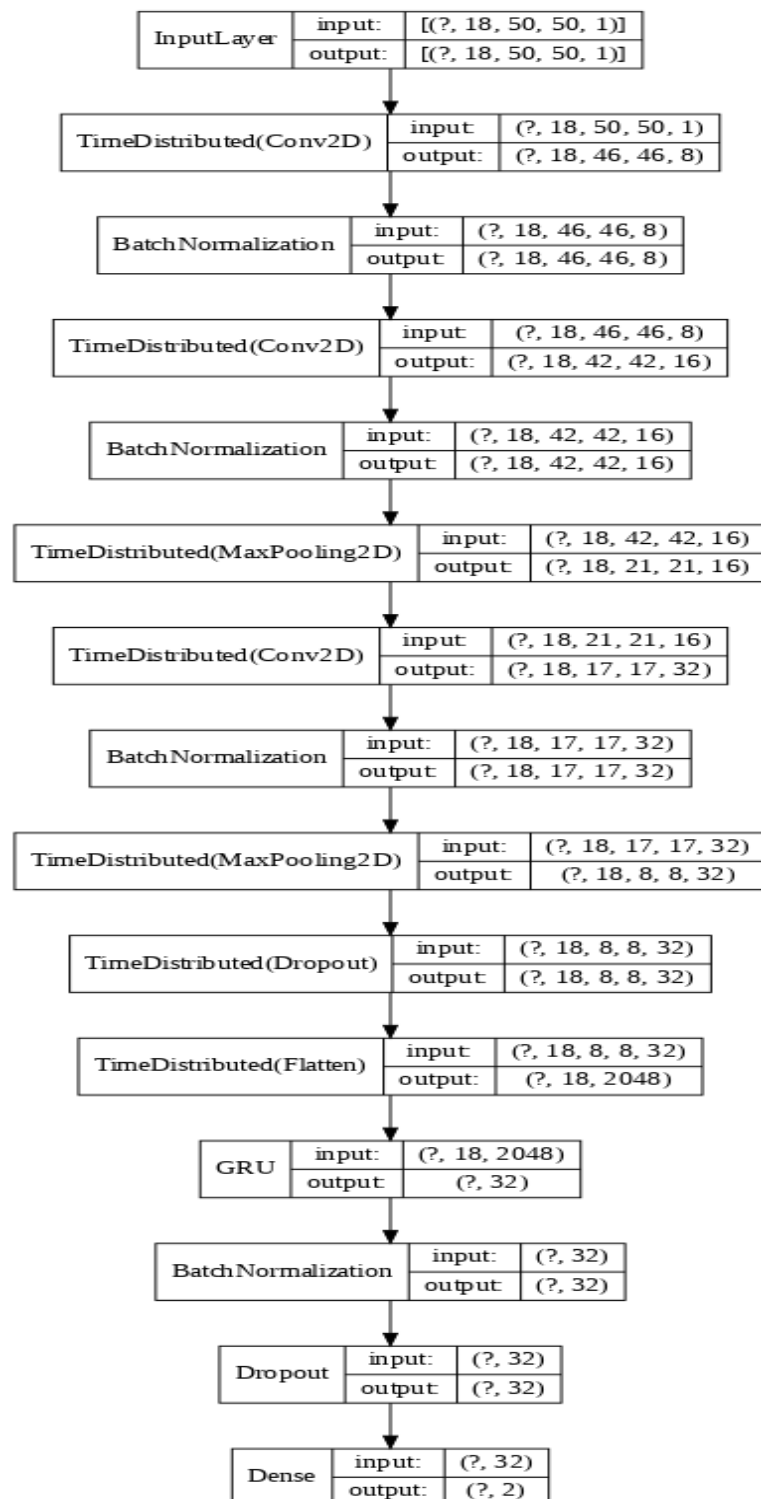


Figura 4.5. Esquema del Modelo de Clasificación..

La configuración de nuestro modelo de clasificación presenta una estructura definida en dos bloques:

- El primer bloque forma parte de una red neuronal convolucional envuelta por una capa TimeDistributed que permite ejecutar en paralelo, al mismo tiempo, dicha red en cada imagen de la secuencia. La arquitectura de esta red neuronal convolucional está compuesta por tres capas convolucionales 2D de 8, 16 y 32 filtros respectivamente, cada una de las cuales capturan patrones visuales más complejos de las imágenes de la secuencia, dos capas Maxpooling resaltan los patrones capturados por las capas convolucionales, tres capas BatchNormalization aceleran el entrenamiento y reducen el impacto del sobreajuste junto a una capa de Abandono del 30%. Por último, una capa flatten transforma la salida tridimensional de los mapas de características de la imagen (altura, ancho, canal) a una entrada unidimensional para el siguiente bloque. Cada capa convolucional de la red adopta una función de activación Relu, un kernel con un tamaño 5×5 y una inicialización de peso glorot_uniform, el cual asigna una distribución de valores a los parámetros del modelo al inicio del entrenamiento.
- El segundo bloque es una red neuronal recurrente formada por una capa GRU que relaciona patrones entre las características espaciales capturadas por la red neuronal convolucional en cada imagen de la secuencia. Esta capa GRU contiene 32 unidades con una función de activación tangente hiperbólica y una inicialización de peso glorot_uniform. A la salida de esta capa GRU le sigue una capa BatchNormalization junto a una regularización L2 de 1e-5 y una capa de Abandono del 10% que juntas intervienen en el ajuste de la red para evitar que no se adapte demasiado a las imágenes del entrenamiento. Al final de la red neuronal recurrente, una capa clasificadora con 2 neuronas, una para cada clase, y una función de activación Softmax elaboran la predicción del modelo de clasificación en términos de probabilidad.

Durante el entrenamiento del modelo, el algoritmo de optimización, Adam, necesita 90 épocas de entrenamiento para aprender cómo debe ajustar 216.360 parámetros a fin de reducir, lo máximo posible, el error predictivo en las imágenes cerebrales PET 3D correspondientes a los 10 grupos de sujetos de prueba. En la Figura 4.6, se ilustra la curva de aprendizaje que refleja el historial del entrenamiento del modelo de clasificación en las 10 iteraciones de la CV Estratificada 10-Fold, así

como el rendimiento obtenido para clasificar los 10 grupos de sujetos de prueba.

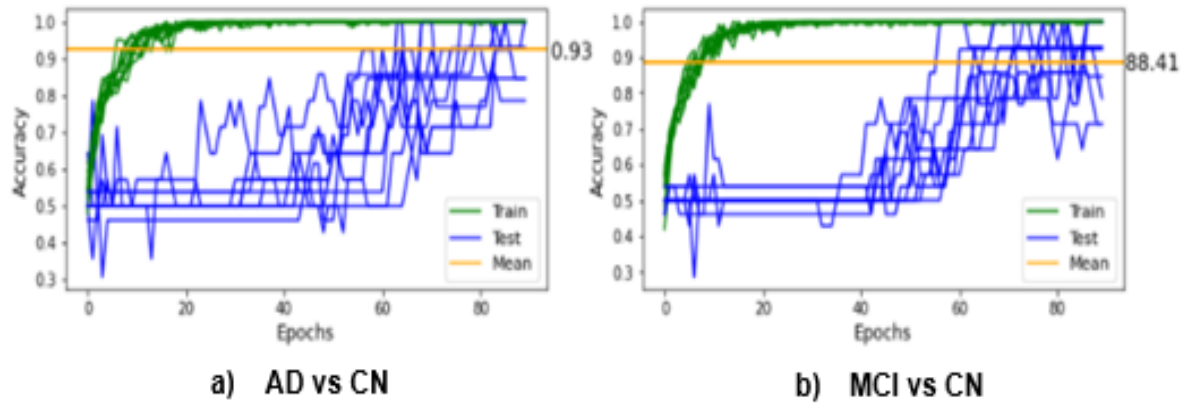


Figura 4.624. Curva de aprendizaje del modelo de clasificación.

Capítulo 5. Interpretación del rendimiento del clasificador

Dada la naturaleza estocástica de los algoritmos de ML, como ya vimos anteriormente en el capítulo 3, el uso de la aleatoriedad en el diseño del modelo de clasificación aumenta su capacidad de aprendizaje pero, a su vez, también se ve comprometida su estabilidad.

En la Figura 5.1 se puede contemplar un claro ejemplo de cómo la aleatoriedad puede hacer que nuestro modelo de clasificación (AD vs CN y MCI vs CN) reproduzca un rendimiento diferente para clasificar un grupo de sujetos de prueba cada vez que se vuelve a entrenar con las mismas imágenes cerebrales.

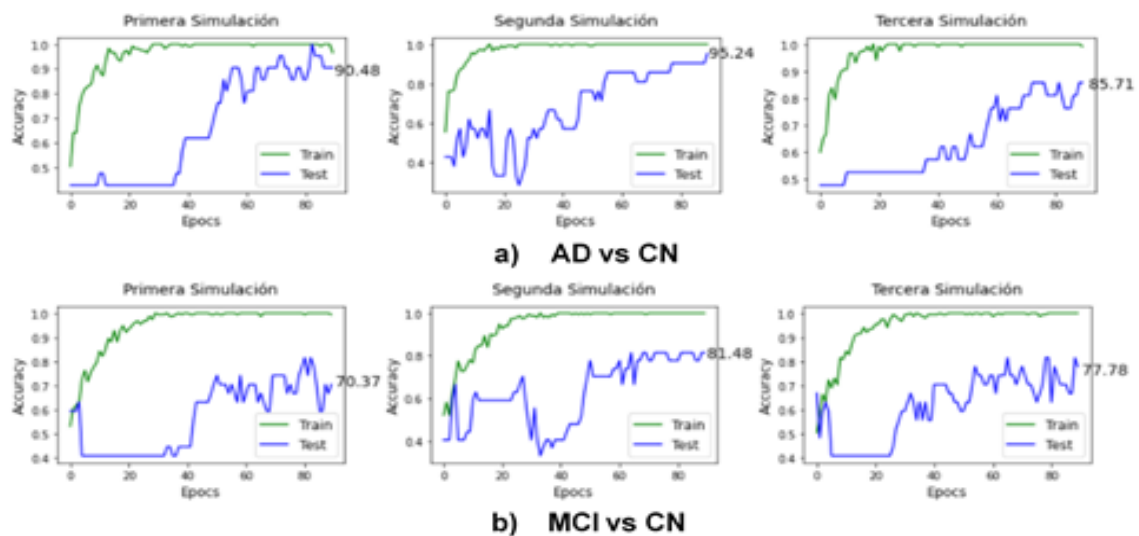


Figura 5.1. Evaluación del Clasificador en un Grupo de Sujetos de Prueba.

Este comportamiento inestable también se puede ver reflejada en la estimación que ofrece la CV Estratificada 10-Fold acerca del rendimiento del modelo para clasificar nuevas imágenes cerebrales PET (Figura 5.2).

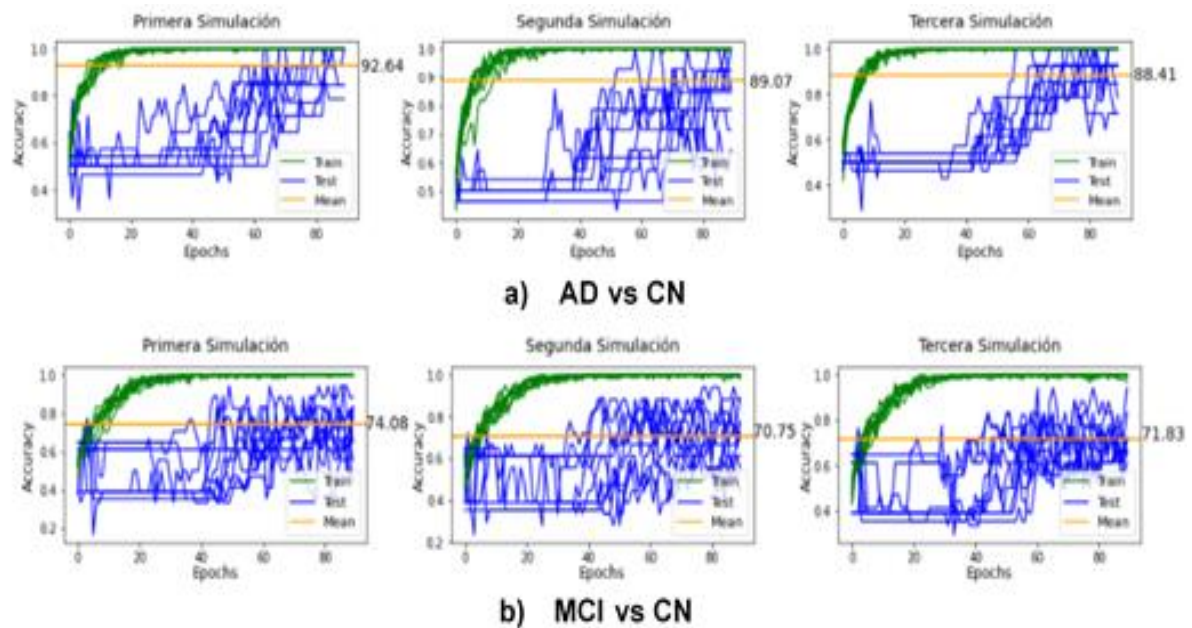


Figura 5.2. Evaluación del Clasificador mediante la CV Estratificada 10-Fold

A priori, esta situación puede llegar a ser muy confuso y dar una sensación de poca credibilidad del rendimiento del modelo de clasificación. Sin embargo, la estabilidad del modelo se puede controlar simplemente con una técnica de conjunto simple. Para ello, se entrena el modelo de clasificación, en múltiples ocasiones, con imágenes cerebrales del mismo grupo de sujetos y se combinan sus predicciones realizadas en imágenes cerebrales de un grupo de sujetos de prueba por medio de una votación por mayoría.

A continuación, en la Figura 5.3 se puede apreciar cómo el rendimiento se va estabilizándose a medida que se incorporan más clasificadores al conjunto.

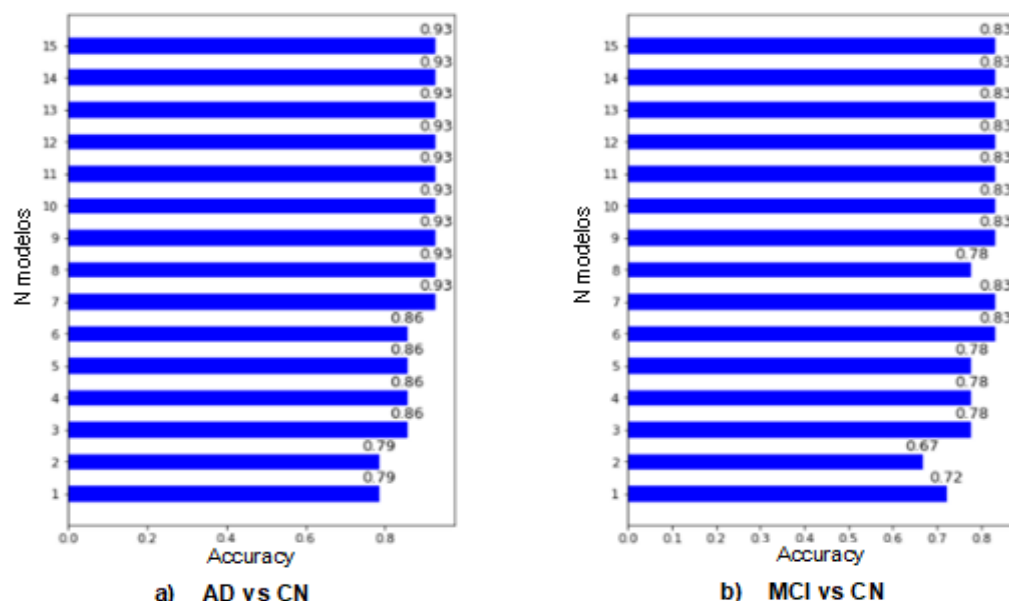


Figura 5.325. Evaluación del Conjunto de Clasificadores en un Grupo de Sujetos de Prueba.

Una vez que se ha garantizado la estabilidad del modelo de clasificación formando un conjunto de 15 clasificadores iguales, se procede a evaluar el rendimiento de dicho conjunto en 10 grupos de prueba diferentes a través de la CV Estratificada 10-Fold. (Figura 5.4).

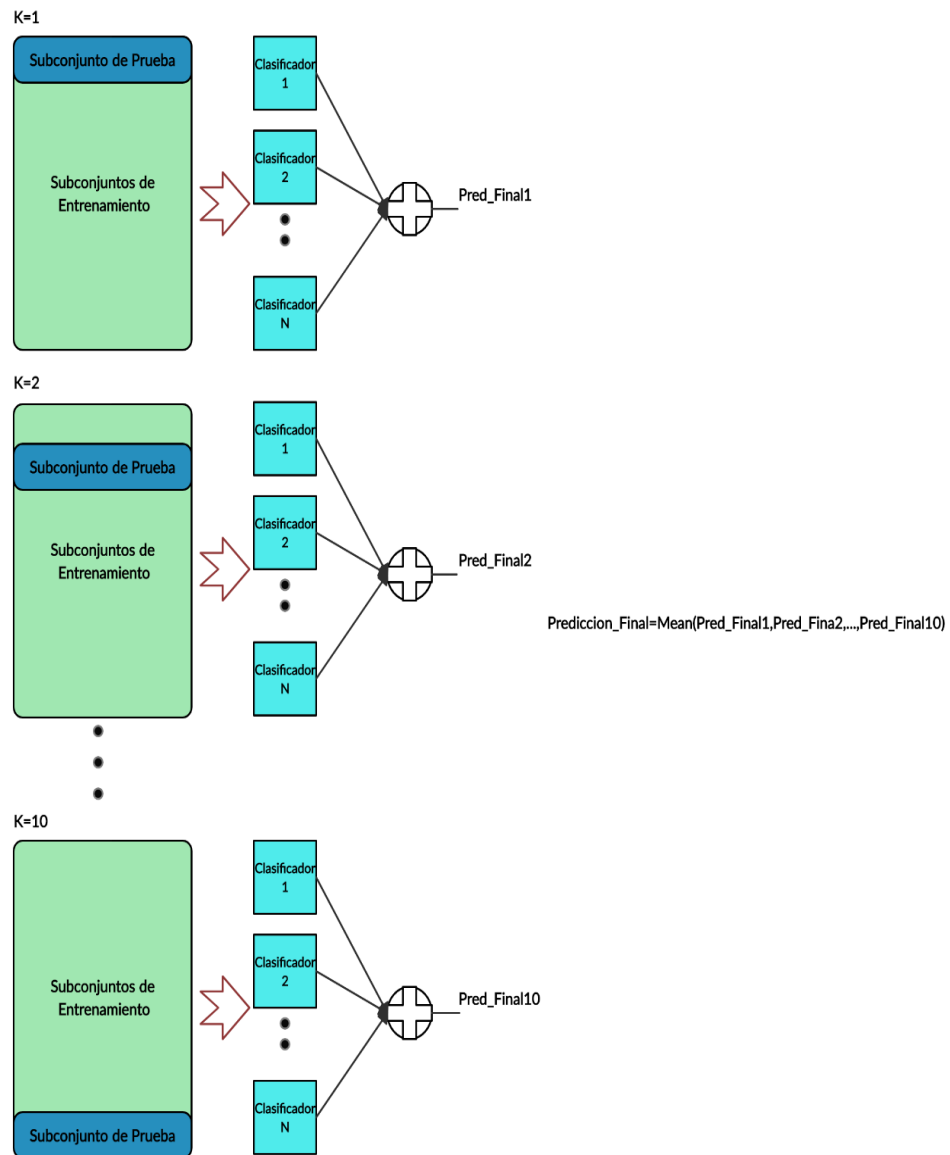


Figura 5.4. Evaluación del Conjunto de Clasificadores mediante la Validación Cruzada Estratificada 10-Fold.

La tabla siguiente (Tabla 5.1) muestra la habilidad del conjunto para clasificar el diagnóstico de la enfermedad en cada grupo de sujetos de prueba, así como el promedio en los 10 grupos.

		AD vs CN (%)				MCI vs CN (%)			
		ACC	SENS	SPEC	AUC	ACC	SENS	SPEC	AUC
K-Fold	k=1	92.85	85.71	100	92.85	61.11	83.33	16.66	66.66
	k=2	100	100	100	100	72.22	100	28.57	66.88
	k=3	78.57	100	57.14	78.57	77.77	100	42.85	64.93
	k=4	100	100	100	100	66.66	90.90	28.57	59.74
	k=5	85.71	85.71	85.71	87.57	77.77	81.81	71.42	78.57
	k=6	100	100	100	100	77.77	90.90	57.14	76.62
	k=7	100	100	100	100	77.77	90.90	57.14	78.57
	k=8	76.92	85.71	66.66	76.19	72.22	90.90	42.85	62.33
	k=9	84.61	71.42	100	85.71	76.47	81.81	66.66	65.15
	k=10	84.61	85.71	83.33	84.52	52.94	54.54	50.00	61.36
	Mean	90.51	91.42	89.55	90.49	71.34	86.51	46.19	68.01

Tabla 5.1. Rendimiento del Conjunto de Clasificadores.

Con el objetivo de tener una visión más clara acerca del rendimiento del conjunto, se exponen la Figura 5.5 y la Figura 5.6.

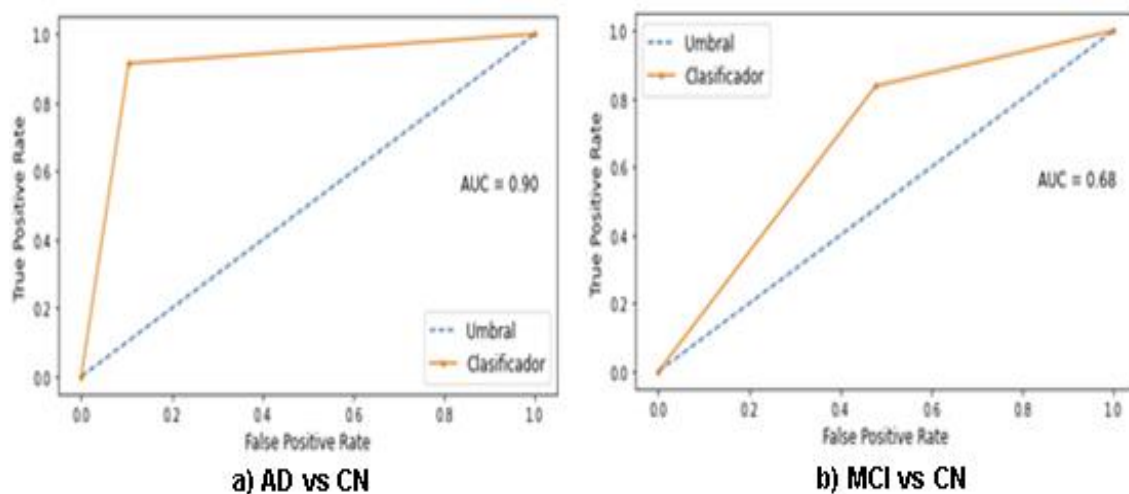


Figura 5.526. Curva ROC del Conjunto de Clasificadores.

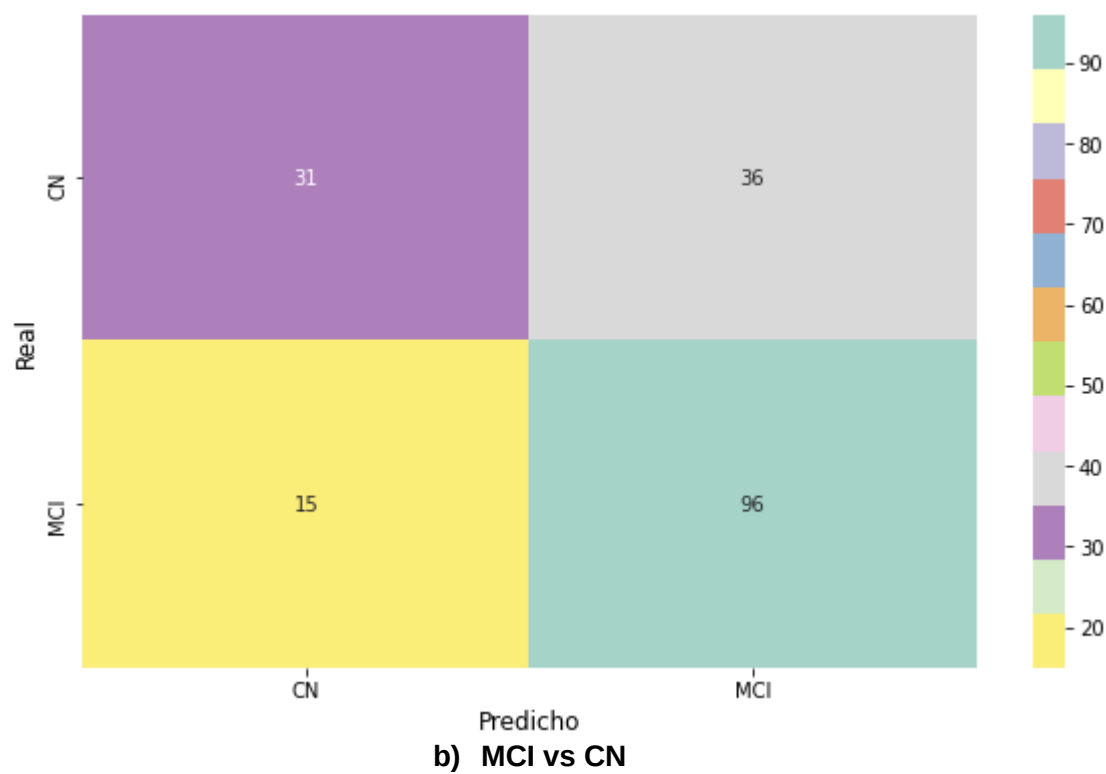
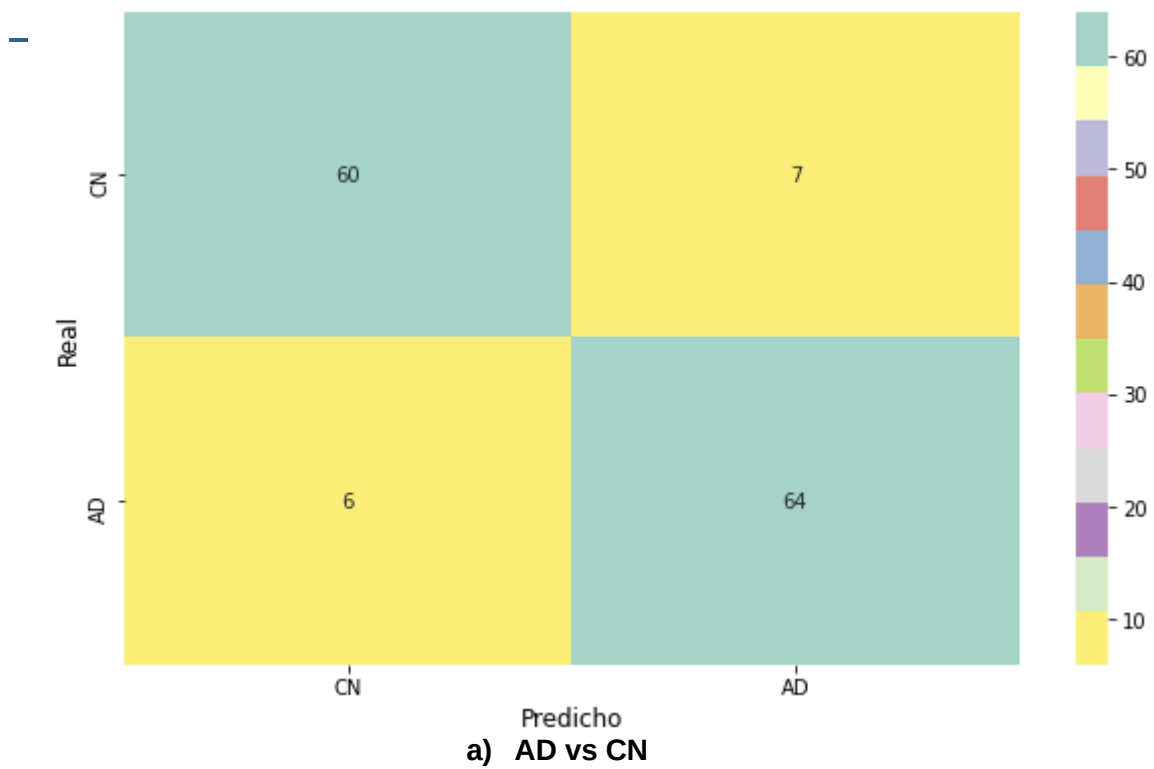


Figura 5.627. Matriz de Confusión del Conjunto de Clasificadores.



Capítulo 6. Conclusiones

La necesidad de encontrar un tratamiento eficaz para tratar la enfermedad de Alzheimer empieza por un diagnóstico temprano. El aprendizaje automático asume este papel logrando ser capaz de predecir e identificar patrones complejos que al ojo del ser humano no parecen ser evidentes bajo la analítica de enormes cantidades de datos [25].

El aprendizaje automático no está para sustituir la función de un médico sino más bien está para ofrecer diagnósticos más rápidos y precisos. Además, esta tecnología puede llevar a cabo tareas monótonas y repetitivas, y centrarnos realmente en problemas mucho más complejos y productivos.

La implementación del aprendizaje automático en el campo de la medicina, en concreto, para aprender a diagnosticar la enfermedad de Alzheimer a través de imágenes cerebrales ha logrado muy buenos resultados teniendo en cuenta la escasez de dichas imágenes de las que se han dispuesto para el estudio [26, 27]:

- En el caso de la clasificación AD vs CN realizada por el conjunto de clasificadores alcanza una sensibilidad del 91.42%. Esto significa que el conjunto detecta la enfermedad de Alzheimer en el 91.42% de los casos, y tan sólo en el 8.58% restante, el conjunto se equivoca prediciendo a un sujeto con un cognitivo normal cuando realmente se corresponde con un sujeto diagnosticado con Alzheimer.
- Por otro lado, en la clasificación MCI vs CN realizada por el conjunto de clasificadores obtiene una sensibilidad del 86.51%. Esto significa que el conjunto detecta a un sujeto que presenta un deterioro cognitivo leve en el 86.51% de los casos, y en el 13.49% restante, el conjunto se equivoca prediciendo a un sujeto con un cognitivo normal cuando realmente se corresponde con un sujeto que padece un deterioro cognitivo leve.

Referencias

[1] “Que es la enfermedad de Alzheimer”, Alzheimer.gov, Documento en formato HTML accesible por internet en la dirección:

<https://www.alzheimers.gov/es/alzheimer-demencias/enfermedad-alzheimer>

[2] “Que es el Alzheimer”, Alzheimer’s association, Documento en formato HTML accesible por internet en la dirección: <https://www.alz.org/alzheimer-demencia/que-es-la-enfermedad-de-alzheimer>

[3] “Inteligencia Artificial Que es IA y Por Qué Importa”, SAS, Documento en formato HTML accesible por internet en la dirección:

https://www.sas.com/es_es/insights/analytics/what-is-artificial-intelligence.html

[4] “Machine Learning: de los datos al modelo”, Innova&acción. Documento en formato HTML accesible por internet en la dirección <https://innovayaccion.com/machine-learning-de-los-datos-al-modelo>

[5] “Qué es el deterioro cognitivo leve”, Fundación Pasqual Maragall, Documento en formato HTML accesible por internet en la dirección <https://blog.fpmaragall.org/que-es-el-deterioro-cognitivo-leve>

[6] “Dementia Statistics, Alzheimer’s Disease International, Documento en formato HTML accesible por internet en la dirección <https://www.alzint.org/about/dementia-facts-figures/dementia-statistics/>

[7] “Cuál es la causa de muerte de un enfermo de Alzheimer”, Know Alzheimer, Documento en formato HTML accesible por internet en la dirección:

<https://knowalzheimer.com/cual-es-la-causa-de-la-muerte-de-un-enfermo-de-alzheimer/>

[8] “Las 10 principales causas de defunción”, Organización Mundial de la Salud, Documento en formato HTML accesible por internet en la dirección: <https://www.who.int/es/news-room/fact-sheets/detail/the-top-10-causes-of-death>

[9] “Alzheimer”, Cuídate Plus, Documento en formato HTML accesible por internet en la dirección:

<https://cuidateplus.marca.com/enfermedades/neurologicas/alzheimer.html>

-
- [10] “Causas y factores de Riesgo”, Alzheimer Association, Documento en formato HTML accesible por internet en la dirección: <https://www.alz.org/alzheimer-demencia/causas-y-factores-de-riesgo?lang=es-MX>
- [11] “Etapas del Alzheimer”, Alzheimer’s Association, Documento en formato HTML accesible por internet en la dirección: <https://www.alz.org/alzheimer-demencia/etapas>
- [12] “Origen del concepto de Inteligencia Artificial”, 2019, Redacción España, Documento en formato HTML accesible por internet en la dirección: <https://www.agenciab12.com/noticia/origen-concepto-inteligencia-artificial>
- [13] J. A. Pascual, “Inteligencia Artificial: qué es, como funciona y para qué se utiliza en la actualidad”, Documento en formato HTML accesible por internet en la dirección: <https://computerhoy.com/reportajes/tecnologia/inteligencia-artificial-469917>
- [14] P. Recuero, “Tipos de aprendizaje en Machine Learning: supervisado y no supervisado”, Documento en formato HTML accesible por internet en la dirección: <https://empresas.blogthinkbig.com/que-algoritmo-elegir-en-ml-aprendizaje/>
- [15] R. Arrabales, “Deep Learning: qué es y por qué va a ser una tecnología clave en el futuro de la inteligencia artificial”, Documento en formato HTML accesible por internet en la dirección: <https://www.xataka.com/robotica-e-ia/deep-learning-que-es-y-por-que-va-a-ser-una-tecnologia-clave-en-el-futuro-de-la-inteligencia-artificial>
- [16] Pablo,” Deep Learning vs Machine Learning: ¿en qué se diferencian?, Blog Orange, Documento en formato HTML accesible por internet en la dirección: <https://blog.orange.es/innovacion/deep-learning-vs-machine-learning/>
- [17] I. Pérez Roldan, “Clasificación de Obras de Arte por Estilo Artístico Usando Redes Neuronales Convolucionales”, Trabajo Fin de Grado, Universidad Politécnica de Madrid (España), 2018.
- [18] D. Calvo, “Función de activación-Redes neuronales”, 2018, Documento en formato HTML accesible por internet en la dirección: <https://www.diegocalvo.es/funcion-de-activacion-redes-neuronales/>

[19] J. Bagnato, “¿Cómo funcionan las Convolutional Neural Networks?”, 2020, Documento en formato HTML accesible por internet en la dirección: <https://www.aprendemachinelearning.com/como-funcionan-las-convolutional-neural-networks-vision-por-ordenador/>

[20] D. Calvo, “Red Neuronal Convolutacional CNN”, 2017, Documento en formato HTML accesible por internet en la dirección: <https://www.diegocalvo.es/red-neuronal-convolutacional/>

[21] “Introducción a las Redes Neuronales Recurrentes”, 2019, Documento en formato HTML accesible por internet en la dirección: <https://www.codificandobits.com/blog/introduccion-redes-neuronales-recurrentes/>

[22] D. Calvo, “Red Neuronal Recurrente – RNN”, 2018, Documento en formato HTML accesible por internet en la dirección: <https://www.diegocalvo.es/red-neuronal-recurrente/>

[23] L. Llamas, “Machine Learning con TensorFlow y Keras en Python”, 2020, Documento en formato HTML accesible por internet en la dirección: <https://www.luisllamas.es/machine-learning-con-tensorflow-y-keras-en-python/>

[24] J. Brownlee, “Overfitting y Underfitting With Machine Learning Algorithms”, Documento en formato HTML accesible por internet en la dirección: <https://machinelearningmastery.com/overfitting-and-underfitting-with-machine-learning-algorithms/>

[25] A. Mishra, “Métricas para evaluar su algoritmo de aprendizaje automático”, Documento en formato HTML accesible por internet en la dirección: <https://towardsdatascience.com/metrics-to-evaluate-your-machine-learning-algorithm-f10ba6e38234>

[26] J. Brownlee, “Embrace Randomness in Machine Learning”, Documento en formato HTML accesible por internet en la dirección: <https://machinelearningmastery.com/randomness-in-machine-learning/>

[27] J. Brownlee, “How to Evaluate the Skill of Deep Learning Models”, Documento en formato HTML accesible por internet en la dirección: <https://machinelearningmastery.com/evaluate-skill-deep-learning-models/>

-
- [28] J. Brownlee, "Evaluate the Performance Of Deep Learning Models in Keras", Documento en formato HTML accesible por internet en la dirección: <https://machinelearningmastery.com/evaluate-performance-deep-learning-models-keras/>
- [29] J. Brownlee, "How to Fix k-Fold Cross-Validation for Imbalanced Classification", Documento en formato HTML accesible por internet en la dirección: <https://machinelearningmastery.com/cross-validation-for-imbalanced-classification/>
- [30] J. Brownlee, "A Gentle Introduction to Ensemble Learning", Documento en formato HTML accesible por internet en la dirección: <https://machinelearningmastery.com/what-is-ensemble-learning/>
- [31] N. Demir, "Ensembles Methods in Machine Learning", Documento en formato HTML accesible por internet en la dirección: <https://www.toptal.com/machine-learning/ensemble-methods-machine-learning>
- [32] "Exploración por tomografía por emisión de positrones – tomografía computada", Documento en formato HTML accesible por internet en la dirección: <https://www.radiologyinfo.org/es/info/pet>
- [33] "Positron Emission Tomography", Johns Hopkins, Documento en formato HTML accesible por internet en la dirección: <https://www.hopkinsmedicine.org/health/treatment-tests-and-therapies/positron-emission-tomography-pet>
- [34] P. Rodríguez-Sahagún, "Aplicación de redes neuronales convolucionales y recurrentes al diagnóstico de autismo a partir de resonancias magnéticas funcionales", Trabajo Fin de Grado, Universidad Politécnica de Madrid (España), febrero, 2018.
- [35] N. Y. Romero, "Cálculo de descriptores en imágenes biomédicas para la predicción precoz del Alzheimer", Trabajo Fin de Grado, Universidad de Málaga (España), 2018.
- [36] L. Manhua, C. Danni, y Y. Weiwu, "Clasificación de la enfermedad de Alzheimer mediante la combinación de redes neuronales convolucionales y recurrentes utilizando imágenes FDG-PET", *Frontiers in Neuroinformatics*, Vol.12, Nº, junio, 2018, pp. 12-35.

[37] A. Ortiz, J. Munilla, M. Martínez-Ibáñez, J. Górriz, J. Ramírez, y D. Salas-González, “Detección de la enfermedad de Parkinson utilizando características basadas en isosuperficies y redes neuronales convolucionales”, *Frontiers in Neuroinformatics*, Vol. 13, Nº, Julio, 2019, pp. 13- 48.

[38] J. Brownlee, “How to Develop a Horizontal Voting Deep Learning Ensemble to Reduce Variance”, Documento en formato HTML accesible por internet en la dirección: <https://machinelearningmastery.com/horizontal-voting-ensemble/>

[39] J. Brownlee, “How to Create a Bagging Ensemble of Deep Learning Models in Keras”, Documento en formato HTML accesible por internet en la dirección: <https://machinelearningmastery.com/how-to-create-a-random-split-cross-validation-and-bagging-ensemble-for-deep-learning-in-keras/>

[40] V. Roman, “Machine Learning: Cómo Desarrollar un Modelo desde Cero”, Documento en formato HTML accesible por internet en la dirección: <https://medium.com/datos-y-ciencia/machine-learning-c%C3%B3mo-desarrollar-un-modelo-desde-cero-cc17654f0d48>

[41] J. Brownlee, “How to Accelerate Learning of Deep Neural Networks With Batch Normalization”, Documento en formato HTML accesible por internet en la dirección: <https://machinelearningmastery.com/how-to-accelerate-learning-of-deep-neural-networks-with-batch-normalization/>

[42] J. Brownlee, “How to Reduce Overfitting With Dropout Regularization in Keras”, Documento en formato HTML accesible por internet en la dirección: <https://machinelearningmastery.com/how-to-reduce-overfitting-with-dropout-regularization-in-keras/>